

2026
시나공

기출문제집

정보처리기사

실기

저작권 안내

이 자료는 시나공 독자들에게 제공하는 자료로써 개인적인 용도로만 사용할 수 있습니다.
허락 없이 복제하거나 다른 매체에 옮겨 실을 수 없으며, 상업적 용도로 사용할 수 없습니다.

핵심 요약

- 1장 요구사항 확인
- 2장 데이터 입·출력 구현
- 3장 데이터 입·출력 구현
- 4장 서버 프로그램 구현
- 5장 인터페이스 구현
- 6장 화면 설계
- 7장 애플리케이션 테스트 관리
- 8장 SQL 응용
- 9장 소프트웨어 개발 보안 구축
- 10장 프로그래밍 언어 활용
- 11장 응용 SW 기초 기술 활용
- 12장 제품 소프트웨어 패키징

1장 요구사항 확인



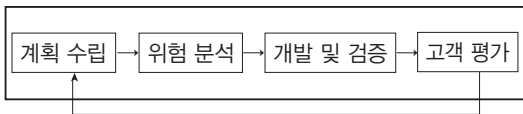
340002 필기 25.8, 25.5, 25.2, 24.5, 23.7, 23.5, 23.2, 22.7, 21.8, 21.3, 20.9, 20.8, 20.6



001 나선형 모형

(B)

- 나선형 모형(Spiral Model, 점진적 모형)은 나선을 따라 돌듯이 여러 번의 소프트웨어 개발 과정을 거쳐 점진적으로 완벽한 최종 소프트웨어를 개발하는 모형이다.
- 보험(Boehm)이 제안하였다.
- 4가지 주요 활동



340003 필기 24.7, 24.2, 21.8, 20.9, 20.8, 20.6



002 폭포수 모형

(B)

- 폭포수 모형(Waterfall Model)은 이전 단계로 돌아갈 수 없다는 전제하에 각 단계를 확실히 매듭짓고 그 결과를 철저하게 검토하여 승인 과정을 거친 후에 다음 단계를 진행하는 개발 방법론이다.
- 가장 오래되고 가장 폭넓게 사용된 전통적인 소프트웨어 생명 주기 모형이다.
- 고전적 생명 주기 모형이라고도 한다.

공급해요 [7]

Q 실기 책에 왜 필기의 기출 년월이 표시되어 있나요?

A 정보처리기사 시험은 필기와 실기가 시험 범위가 같습니다. 동일한 내용이 객관식으로 필기시험에 나올 수도 있고, 단답형이나 서술식으로 실기시험에 나올 수도 있습니다. 공부하다 보면 알겠지만 필기시험과 실기시험에 중복해서 나온 내용이 많습니다. 자격 시험에는 나온 문제가 또 나온다는 걸 명심하세요.

340005

20.7, 필기 25.8, 23.7, 22.4, 22.3, 21.8, 21.5, 20.8



003 애자일 모형

(A)

- 애자일(Agile)은 '민첩한', '기민한'이라는 의미로, 고객의 요구사항 변화에 유연하게 대응할 수 있도록 일정한 주기를 반복하면서 개발하는 모형이다.
- 어느 특정 개발 방법론이 아니라 좋은 것을 빠르고 낭비 없게 만들기 위해 고객과의 소통에 초점을 맞춘 방법론을 통칭한다.
- 대표적인 개발 모형
 - 스크럼(Scrum)
 - XP(eXtreme Programming)
 - 칸반(Kanban)
 - Lean
 - 기능 중심 개발(FDD; Feature Driven Development)

340006

필기 25.8, 24.7, 23.2, 22.3, 21.3, 20.8



004 애자일 개발 4가지 핵심 가치

(B)

- 프로세스와 도구보다는 개인과 상호작용에 더 가치를 둔다.
- 방대한 문서보다는 실행되는 SW에 더 가치를 둔다.
- 계약 협상보다는 고객과 협업에 더 가치를 둔다.
- 계획을 따르기 보다는 변화에 반응하는 것에 더 가치를 둔다.

340010

필기 25.8, 25.2, 24.5, 24.2, 23.7, 23.5, 23.2, 22.7, 22.4, 20.9, 20.6



005 XP



- XP(eXtreme Programming)는 수시로 발생하는 고객의 요구사항에 유연하게 대응하기 위해 고객의 참여와 개발 과정의 반복을 극대화하여 개발 생산성을 향상시키는 방법이다.
- 짧고 반복적인 개발 주기, 단순한 설계, 고객의 적극적인 참여를 통해 소프트웨어를 빠르게 개발하는 것을 목적으로 한다.
- XP의 5가지 핵심 가치
 - 의사소통(Communication)
 - 단순성(Simplicity)
 - 용기(Courage)
 - 존중(Respect)
 - 피드백(Feedback)

440009

20.10, 필기 24.7, 24.5, 22.4, 20.9



006 XP의 주요 실천 방법(Practice)



필기 20.9

Pair Programming(짝 프로그래밍)

다른 사람과 함께 프로그래밍을 수행함으로써 개발에 대한 책임을 공동으로 나눠 갖는 환경을 조성한다.

필기 20.9

Collective Ownership(공동 코드 소유)

개발 코드에 대한 권한과 책임을 공동으로 소유한다.

Test-Driven Development(테스트 주도 개발)

- 개발자가 실제 코드를 작성하기 전에 테스트 케이스를 먼저 작성하므로 자신이 무엇을 해야할지를 정확히 파악한다.
- 테스트가 지속적으로 진행될 수 있도록 자동화된 테스트 도구(구조, 프레임워크)를 사용한다.

Whole Team(전체 팀)

개발에 참여하는 모든 구성원(고객 포함)들은 각자 자신의 역할이 있고 그 역할에 대한 책임을 가져야 한다.

필기 20.9

Continuous Integration(계속적인 통합)

모듈 단위로 나눠서 개발된 코드들은 하나의 작업이 마무리될 때마다 지속적으로 통합된다.

20.10, 필기 24.7, 24.5, 22.4

Refactoring(리팩토링)

- 프로그램 기능의 변경 없이 시스템을 재구성한다.
- 목적 : 프로그램을 쉽게 이해하고 쉽게 수정하여 빠르게 개발할 수 있도록 하기 위함임

Small Releases(소규모 릴리즈)

릴리즈 기간을 짧게 반복함으로써 고객의 요구 변화에 신속히 대응할 수 있다.

340016

21.4, 필기 22.4



007 기능 요구사항



- 기능 요구사항(Functional Requirements)은 시스템이 무엇을 하는지, 어떤 기능을 하는지 등의 기능이나 수행과 관련된 요구사항이다.
- 시스템의 입력이나 출력으로 무엇이 포함되어야 하는지에 대한 사항
- 시스템이 어떤 데이터를 저장하거나 연산을 수행해야 하는지에 대한 사항
- 시스템이 반드시 수행해야 하는 기능
- 사용자가 시스템을 통해 제공받기를 원하는 기능

340017

21.4, 필기 25.5, 24.5, 23.2, 22.4, 21.8



008 비기능 요구사항



- 비기능 요구사항(Non-functional Requirements)은 품질이나 제약사항과 관련된 요구사항이다.
- 시스템 장비 구성 요구사항
- 성능 요구사항
- 인터페이스 요구사항
- 데이터를 구축하기 위해 필요한 요구사항
- 테스트 요구사항
- 보안 요구사항
- 품질 요구사항 : 가용성, 정합성, 상호 호환성, 대응성, 이식성, 확장성, 보안성 등



009 요구사항 분석

C

- 요구사항 분석(Requirement Analysis)은 소프트웨어 개발의 실제적인 첫 단계로, 개발 대상에 대한 사용자의 요구사항을 이해하고 문서화하는 활동을 의미한다.
- 사용자 요구의 타당성을 조사하고 비용과 일정에 대한 제약을 설정한다.
- 사용자의 요구를 정확하게 추출하여 목표를 정한다.



010 자료 흐름도의 구성 요소

A

기호	의미
필기 25.5, 25.2, 24.5, 23.2, 22.7, ... 프로세스(Process)	자료를 변환시키는 시스템의 한 부분(처리 과정)을 나타내며 처리, 기능, 변환, 버블이라고도함
필기 25.5, 25.2, 24.5, 23.2, 22.7, ... 자료 흐름(Data Flow)	자료의 이동(흐름)이나 연관관계를 나타냄
필기 25.5, 25.2, 24.5, 23.2, 22.7, ... 자료 저장소(Data Store)	시스템에서의 자료 저장소(파일, 데이터베이스)를 나타냄
필기 25.5, 24.5, 23.2, 22.7, 22.3, ... 단말(Terminator)	시스템과 교신하는 외부 개체로, 입력 데이터가 만들어지고 출력 데이터를 받음



011 HIPO

B

- HIPO(Hierarchy Input Process Output)는 시스템의 분석 및 설계, 또는 문서화에 사용되는 기법으로, 시스템 실행 과정인 입력·처리·출력의 기능을 표현한 것이다.
- 하향식 소프트웨어 개발을 위한 문서화 도구이다.
- 시스템의 기능을 여러 개의 고위 모듈로 분할하여 이들 간의 인터페이스를 계층 구조로 표현한 것을 HIPO Chart라고 한다.



012 연관 관계

A

- 연관(Association) 관계는 2개 이상의 사물이 서로 관련되어 있는 관계이다.
- 사물 사이를 실선으로 연결하여 표현한다.
- 방향성은 화살표로 표현한다.
- 양방향 관계의 경우 화살표를 생략하고 실선으로만 연결한다.
- 다중도를 선 위에 표기한다.

예제 사람이 집을 소유하는 관계이다. 사람은 자기가 소유하고 있는 집에 대해 알고 있지만 집은 누구에 의해 자신 이 소유되고 있는지 모른다는 의미이다.



013 집합 관계

A

- 집합(Aggregation) 관계는 하나의 사물이 다른 사물에 포함되어 있는 관계이다.
- 포함하는 쪽(전체, Whole)과 포함되는 쪽(부분, Part)은 서로 독립적이다.
- 포함되는 쪽(부분, Part)에서 포함하는 쪽(전체, Whole)으로 속이 빈 마름모를 연결하여 표현한다.

예제 프린터는 컴퓨터에 연결해서 사용할 수 있으며, 다른 컴퓨터에 연결해서 사용할 수도 있다.



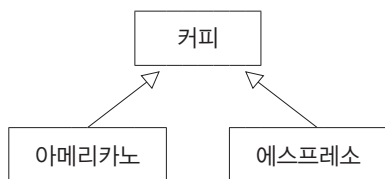


014 일반화 관계

(A)

- 일반화(Generalization) 관계는 하나의 사물이 다른 사물에 비해 더 일반적이거나 구체적인 관계이다.
- 보다 일반적인 개념을 상위(부모), 보다 구체적인 개념을 하위(자식)라고 부른다.
- 구체적(하위)인 사물에서 일반적(상위)인 사물 쪽으로 속이 빈 화살표를 연결하여 표현한다.

예제 아메리카노와 에스프레소는 커피이다. 다시 말하면, 커피에는 아메리카노와 에스프레소가 있다.



015 의존 관계

(A)

- 의존(Dependency) 관계는 연관 관계와 같이 사물 사이에 서로 연관은 있으나 필요에 의해 서로에게 영향을 주는 짧은 시간 동안만 연관을 유지하는 관계이다.
- 하나의 사물과 다른 사물이 소유 관계는 아니지만 사물의 변화가 다른 사물에도 영향을 미치는 관계이다.
- 일반적으로 한 클래스가 다른 클래스를 오퍼레이션의 매개 변수로 사용하는 경우에 나타나는 관계이다.
- 영향을 받는 사물(이용자)이 영향을 주는 사물(제공자) 쪽으로 점선 화살표를 연결하여 표현한다.

예제 등급이 높으면 할인율을 적용하고, 등급이 낮으면 할인율을 적용하지 않는다.



016 구조적 다이어그램의 종류

(B)

필기 25.8, 23.2, 21.5, 21.3, 20.6

클래스 다이어그램(Class Diagram)

클래스와 클래스가 가지는 속성, 클래스 사이의 관계를 표현한다.

필기 20.6

객체 다이어그램(Object Diagram)

- 클래스에 속한 사물(객체)들, 즉 인스턴스(Instance)를 특정 시점의 객체와 객체 사이의 관계로 표현한다.
- 럼바우(Rumbaugh) 객체지향 분석 기법에서 객체 모델링에 활용된다.

필기 23.5, 22.3, 20.6

컴포넌트 다이어그램(Component Diagram)

- 실제 구현 모듈인 컴포넌트 간의 관계나 컴포넌트 간의 인터페이스를 표현한다.
- 구현 단계에서 사용된다.

필기 22.3

배치 다이어그램(Deployment Diagram)

- 결과물, 프로세스, 컴포넌트 등 물리적 요소들의 위치를 표현한다.
- 구현 단계에서 사용된다.

복합체 구조 다이어그램(Composite Structure Diagram)

클래스나 컴포넌트가 복합 구조를 갖는 경우 그 내부 구조를 표현한다.

필기 22.3

패키지 다이어그램(Package Diagram)

유스케이스나 클래스 등의 모델 요소들을 그룹화한 패키지들의 관계를 표현한다.



017 행위 다이어그램의 종류



필기 23.5, 20.8

유스케이스 다이어그램(Use Case Diagram)

- 사용자의 요구를 분석하는 것으로, 기능 모델링 작업에 사용한다.
- 사용자(Actor)와 사용 사례(Use Case)로 구성된다.

필기 25.8, 23.5, 23.2, 22.3, 21.5, 21.3, 20.8

순차 다이어그램(Sequence Diagram)

상호작용하는 시스템이나 객체들이 주고받는 메시지를 표현한다.

커뮤니케이션 다이어그램(Communication Diagram)

동작에 참여하는 객체들이 주고받는 메시지와 객체들 간의 연관 관계를 표현한다.

필기 25.8, 23.5, 21.3, 20.9

상태 다이어그램(State Diagram)

- 하나의 객체가 자신이 속한 클래스의 상태 변화 혹은 다른 객체와의 상호작용에 따라 상태가 어떻게 변화하는지를 표현한다.
- 럼바우(Rumbaugh) 객체지향 분석 기법에서 동적 모델링에 활용된다.

필기 25.8, 23.2, 21.5, 20.8

활동 다이어그램(Activity Diagram)

시스템이 어떤 기능을 수행하는지 객체의 처리 로직이나 조건에 따른 처리의 흐름을 순서에 따라 표현한다.

상호작용 개요 다이어그램(Interaction Overview Diagram)

상호작용 다이어그램 간의 제어 흐름을 표현한다.

타이밍 다이어그램(Timing Diagram)

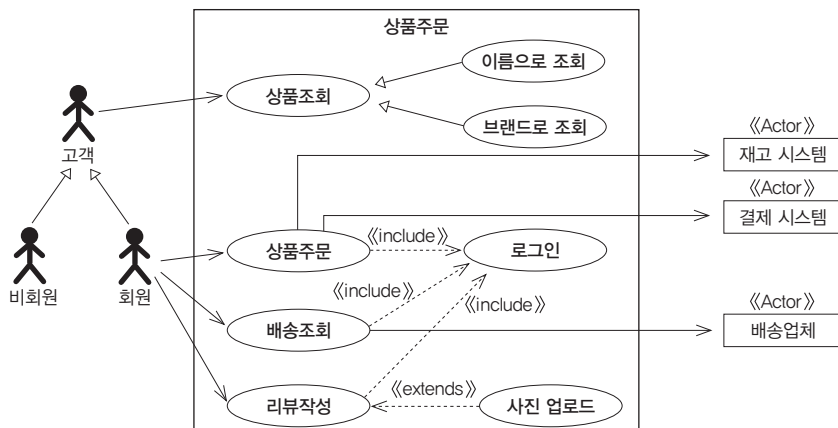
객체 상태 변화와 시간 제약을 명시적으로 표현한다.



- 유스케이스(Use Case) 다이어그램은 사용자와 다른 외부 시스템들이 개발될 시스템을 이용해 수행할 수 있는 기능을 사용자의 관점에서 표현한 것이다.
- 유스케이스 다이어그램의 구성 요소

구성 요소	표현 방법	내용
시스템(System) / 시스템 범위(System Scope)	상품주문	시스템 내부의 유스케이스들을 사각형으로 묶어 시스템의 범위를 표현한 것
필기 21.5 액터(Actor)	• 주액터  고객 • 부액터 <<Actor>> 재고 시스템	• 시스템과 상호작용을 하는 모든 외부 요소 • 주로 사람이나 외부 시스템을 의미함 • 주액터 : 시스템을 사용함으로써 이득을 얻는 대상으로, 주로 사람이 해당됨 • 부액터 : 주액터의 목적 달성을 위해 시스템에 서비스를 제공하는 외부 시스템으로, 조직이나 기관 등이 될 수 있음
유스케이스(Use Case)	상품조회	사용자가 보는 관점에서 시스템이 액터에게 제공하는 서비스나 기능을 표현한 것
관계(Relationship)	• 포함 <<include>>  • 확장 <<extends>>  • 일반화 	• 유스케이스 다이어그램에서 관계는 액터와 유스케이스, 유스케이스와 유스케이스 사이에서 나타날 수 있음 • 유스케이스에서 나타날 수 있는 관계 : 포함(Include) 관계, 확장(Extends) 관계, 일반화(Generalization) 관계

예제 다음은 회원과 비회원으로 구분되는 회원이 상품주문 과정에서 수행할 수 있는 기능을 표현한 유스케이스 다이어그램이다.





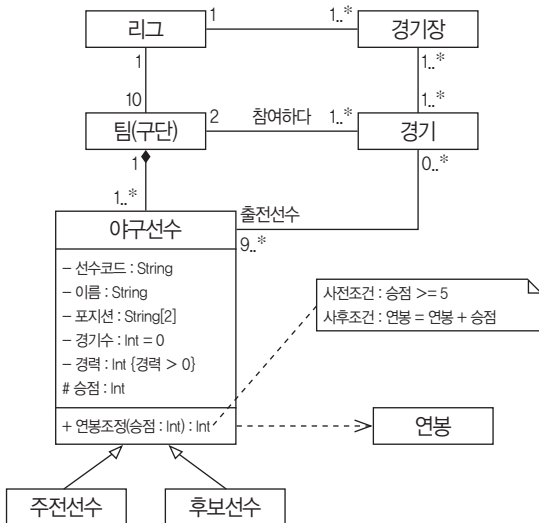
019 클래스 다이어그램



- 클래스(Class) 다이어그램은 클래스와 클래스가 가지는 속성, 클래스 사이의 관계를 표현한 것이다.
- 클래스 다이어그램의 구성 요소

구성 요소	표현 방법	내용
21.10, 필기 21.8 클래스(Class)	클래스명 속성1 속성2 오퍼레이션1 오퍼레이션2	• 각각의 객체들이 갖는 속성과 오퍼레이션(동작)을 표현한 것 • 일반적으로 3개의 구획(Compartment)으로 나뉘 클래스의 이름, 속성, 오퍼레이션을 표기함 • 속성(Attribute) : 클래스의 상태나 정보를 표현함 • 오퍼레이션(Operation) : 클래스가 수행할 수 있는 동작으로, 함수(메소드, Method)라고도 함
제약조건		• 속성에 입력될 값에 대한 제약조건이나 오퍼레이션 수행 전후에 지정해야 할 조건이 있다면 이를 적음 • 클래스 안에 제약조건을 기술할 때는 중괄호 { }를 이용함
관계(Relationships)		• 관계는 클래스와 클래스 사이의 연관성을 표현함 • 클래스 다이어그램에 표현하는 관계에는 연관 관계, 집합 관계, 포함 관계, 일반화 관계, 의존 관계가 있음

예제 다음은 프로야구 리그에 필요한 정보의 일부를 표현한 클래스 다이어그램이다.

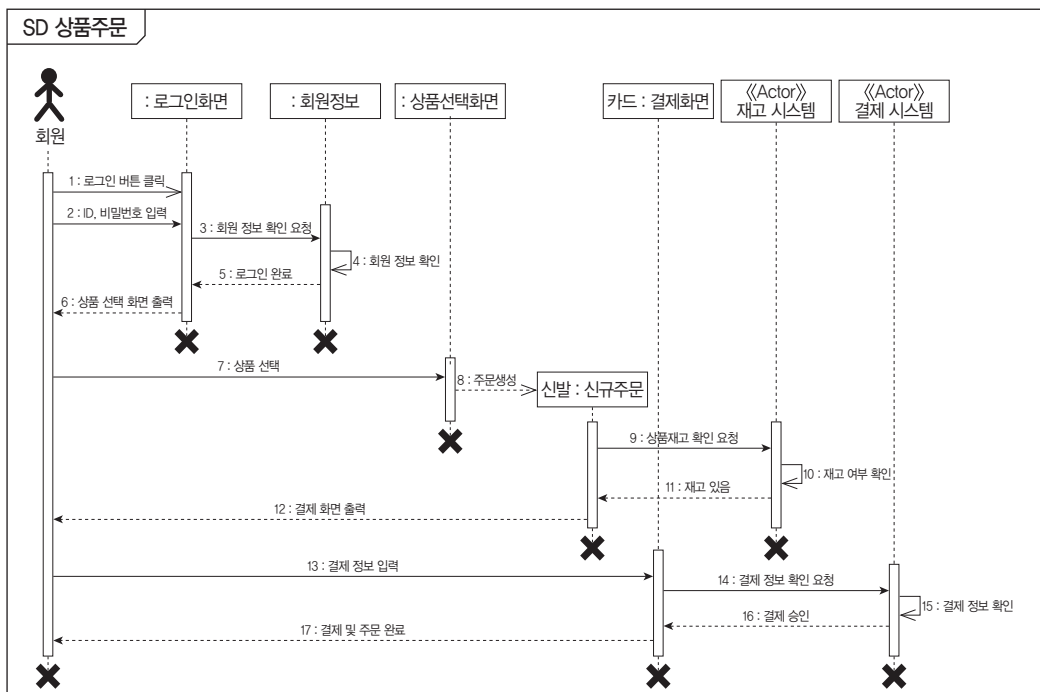




- 순차(Sequence) 다이어그램은 시스템이나 객체들이 메시지를 주고받으며 상호작용하는 과정을 그림으로 표현한 것이다.
- 순차 다이어그램의 구성 요소

구성 요소	표현 방법	의미
액터(Actor)	 회원	시스템으로부터 서비스를 요청하는 외부 요소로, 사람이나 외부 시스템을 의미함
객체(Object)	: 로그인화면	메시지를 주고받는 주체
필기 22.7, 20.8 생명선(Lifeline)		• 객체가 메모리에 존재하는 기간으로, 객체 아래쪽에 점선을 그어 표현함 • 객체 소멸(×)이 표시된 기간까지 존재함
필기 22.7, 20.8 실행 상자 (Active Box, 활성 상자)		객체가 메시지를 주고받으며 구동되고 있음을 표현함
필기 22.7, 20.8 메시지(Message)	1: 로그인 버튼 클릭 →	객체가 상호 작용을 위해 주고받는 메시지
객체 소멸	×	해당 객체가 더 이상 메모리에 존재하지 않음을 표현한 것
프레임(Frame)	SD 상품주문	다이어그램의 전체 또는 일부를 묶어 표현한 것

예제 다음은 회원의 상품 주문 과정에 재고 시스템과 결제 시스템이 관계된 상호 작용 과정을 표현한 순차 다이어그램이다.





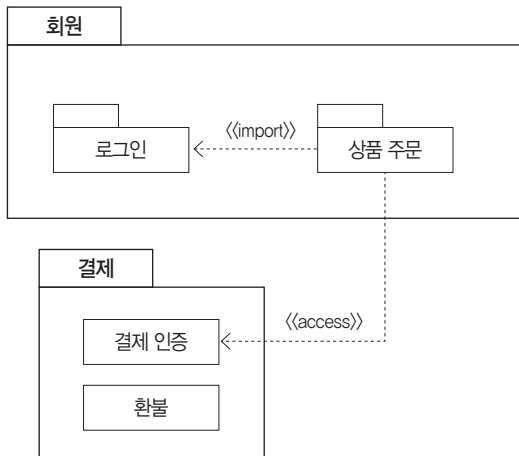
021 패키지 다이어그램



- 패키지(Package) 다이어그램은 유스케이스나 클래스 등의 요소들을 그룹화한 패키지 간의 의존 관계를 표현한 것이다.
- 패키지 다이어그램의 구성 요소

구성 요소	표현 방법	의미
패키지(Package)		- 객체들을 그룹화한 것 - 단순 표기법 : 패키지 안에 패키지 이름만 표현 - 확장 표기법 : 패키지 안에 요소까지 표현
객체(Object)		유스케이스, 클래스, 인터페이스, 테이블 등 패키지에 포함될 수 있는 다양한 요소들
의존 관계 (Dependency)		- 패키지와 패키지, 패키지와 객체 간을 점선 화살표로 연결하여 표현함 - 스테레오 타입을 이용해 의존 관계를 구체적으로 표현할 수 있음 - 의존 관계의 표현 형태는 사용자가 임의로 작성할 수 있으며, 대표적으로 import와 access가 사용됨 《import》: 패키지에 포함된 객체들을 직접 가져와서 이용하는 관계 《access》: 인터페이스를 통해 패키지 내의 객체에 접근하여 이용하는 관계

예제 다음은 회원이 상품 주문 시 사용하는 〈회원〉, 〈로그인〉, 〈상품 주문〉, 〈결제〉 패키지들 간의 의존 관계를 표현한 패키지 다이어그램이다.





022 컴포넌트 기반 방법론

(C)

- 컴포넌트 기반 방법론(CBD; Component Based Design)은 기존의 시스템이나 소프트웨어를 구성하는 컴포넌트를 조합하여 하나의 새로운 애플리케이션을 만드는 방법론이다.
- 컴포넌트의 재사용(Reusability)이 가능하여 시간과 노력을 절감할 수 있다.
- 컴포넌트 기반 방법론의 개발 절차



023 CASE

(B)

- CASE(Computer Aided Software Engineering)는 소프트웨어 개발 과정에서 사용되는 요구 분석, 설계, 구현, 검사 및 디버깅 과정 전체 또는 일부를 컴퓨터와 전용 소프트웨어 도구를 사용하여 자동화하는 것이다.
- CASE의 주요 기능
 - 소프트웨어 생명 주기 전 단계의 연결
 - 다양한 소프트웨어 개발 모형 지원
 - 그래픽 지원



024 LOC 기법

(A)

- LOC(source Line Of Code, 원시 코드 라인 수) 기법은 소프트웨어 각 기능의 원시 코드 라인 수의 비관치, 낙관치, 기대치를 측정하여 예측치를 구하고 이를 이용하여 비용을 산정하는 기법이다.
- 산정 공식
 - 노력(인월) = 개발 기간 × 투입 인원

$$= LOC / 1인당 월평균 생산 코드 라인 수$$
 - 개발 비용

$$= 노력(인월) \times 단위 비용(1인당 월평균 인건비)$$
 - 개발 기간 = 노력(인월) / 투입 인원
 - 생산성 = LOC / 노력(인월)



025 수학적 산정 기법

(C)

- 수학적 산정 기법은 상향식 비용 산정 기법으로, 경험적 추정 모형, 실험적 추정 모형이라고도 한다.
- 수학적 산정 기법은 개발 비용 산정의 자동화를 목표로 한다.
- 주요 수학적 산정 기법
 - COCOMO 모형
 - Putnam 모형
 - 기능 점수(FP) 모형



026 COCOMO 모형

(C)

- COCOMO(CONstructive COst MOdel) 모형은 원시 프로그램의 규모인 LOC(원시 코드 라인 수)에 의한 비용 산정 기법이다.
- 개발할 소프트웨어의 규모(LOC)를 예측한 후 이를 소프트웨어 종류에 따라 다르게 책정되는 비용 산정 방정식에 대입하여 비용을 산정한다.
- 비용 산정 결과는 프로젝트를 완성하는 데 필요한 노력(Man-Month)으로 나타난다.
- 보헴(Boehm)이 제안하였다.



027 COCOMO의 소프트웨어 개발 유형

(B)

필기 23.5, 23.2, 22.4, 21.8, 21.5, 21.3, 20.8, 20.6

조직형(Organic Mode)

- 기관 내부에서 개발된 중·소 규모의 소프트웨어이다.
- 일괄 자료 처리나 과학기술 계산용, 비즈니스 자료 처리용 등의 5만(50KDSI) 라인 이하의 소프트웨어를 개발하는 유형이다.
- 사무 처리용, 업무용, 과학용 응용 소프트웨어 개발에 적합하다.

반분리형(Semi-Detached Mode)

- 조직형과 내장형의 중간형 소프트웨어이다.
- 트랜잭션 처리 시스템이나 운영체제, 데이터베이스 관리 시스템 등의 30만(300KDSI) 라인 이하의 소프트웨어를 개발하는 유형이다.
- 컴파일러, 인터프리터와 같은 유틸리티 개발에 적합하다.

내장형(Embedded Mode)

- 초대형 규모의 소프트웨어이다.
- 트랜잭션 처리 시스템이나 운영체제 등의 30만(300KDSI) 라인 이상의 소프트웨어를 개발하는 유형이다.
- 신호기 제어 시스템, 미사일 유도 시스템, 실시간 처리 시스템 등의 시스템 프로그램 개발에 적합하다.



028 Putnam 모형

ⓒ

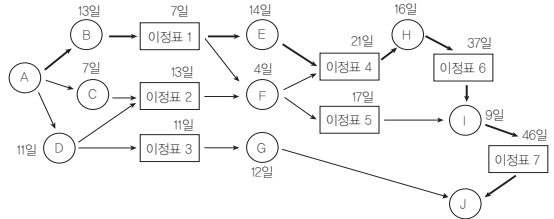
- Putnam 모형은 소프트웨어 생명 주기의 전 과정 동안에 사용될 노력의 분포를 예상하는 모형이다.
- 푸트남(Putnam)이 제안한 것으로, 생명 주기 예측 모형이라고도 한다.
- 시간에 따른 함수로 표현되는 Rayleigh-Norden 곡선의 노력 분포도를 기초로 한다.



029 CPM

Ⓑ

- CPM(Critical Path Method, 임계 경로 기법)은 프로젝트 완성에 필요한 작업을 나열하고 작업에 필요한 소요 기간을 예측하는데 사용하는 기법이다.
- CPM은 노드와 간선으로 구성된 네트워크로 노드는 작업을, 간선은 작업 사이의 전후 의존 관계를 나타낸다.



※ 그림에서 굵은선이 임계 경로, 즉 최장 경로입니다.



030 CMMI

Ⓑ

- CMMI(Capability Maturity Model Integration)는 소프트웨어 개발 조직의 업무 능력 및 조직의 성숙도를 평가하는 모델이다.
- CMMI의 소프트웨어 프로세스 성숙도

단계	특징
초기(Initial)	작업자 능력에 따라 성공 여부 결정
필기 23.2, 20.9, 20.6 관리(Managed)	특정한 프로젝트 내의 프로세스 정의 및 수행
필기 23.2, 20.9, 20.6 정의(Defined)	조직의 표준 프로세스를 활용하여 업무 수행
정량적 관리 (Quantitatively Managed)	프로젝트를 정량적으로 관리 및 통제
필기 20.9, 20.6 최적화 (Optimizing)	프로세스 역량 향상을 위해 지속적인 프로세스 개선



031 SPICE



- SPICE(Software Process Improvement and Capability dEtermination)는 정보 시스템 분야에서 소프트웨어의 품질 및 생산성 향상을 위해 소프트웨어 프로세스를 평가 및 개선하는 국제 표준이다.
- 공식 명칭은 ISO/IEC 15504이다.



032 소프트웨어 개발 프레임워크



- 소프트웨어 개발 프레임워크(Framework)는 소프트웨어 개발에 공통적으로 사용되는 구성 요소와 아키텍처를 일반화하여 손쉽게 구현할 수 있도록 여러 가지 기능들을 제공해주는 반제품 형태의 소프트웨어 시스템이다.
- 선행 사업자의 기술에 의존하지 않는 표준화된 개발 기반으로 인해 사업자 종속성이 해소된다.



033 소프트웨어 개발 프레임워크의 특성



필기 22.7, 21.5, 20.9, 20.6

모듈화(Modularity)

- 프레임워크는 캡슐화를 통해 모듈화를 강화하고 설계 및 구현의 변경에 따른 영향을 최소화함으로써 소프트웨어의 품질을 향상시킨다.
- 프레임워크는 개발 표준에 의한 모듈화로 인해 유지보수가 용이하다.

필기 22.7, 21.5, 20.9, 20.6

재사용성(Reusability)

프레임워크는 재사용 가능한 모듈들을 제공함으로써 예산 절감, 생산성 향상, 품질 보증이 가능하다.

필기 22.7, 21.5

확장성(Extensibility)

프레임워크는 다형성(Polymorphism)을 통한 인터페이스 확장이 가능하여 다양한 형태와 기능을 가진 애플리케이션 개발이 가능하다.

제어의 역흐름(Inversion of Control)

개발자가 관리하고 통제해야 하는 객체들의 제어를 프레임워크에 넘김으로써 생산성을 향상시킨다.

2장 데이터 입·출력 구현



440063



034 스키마

(A)

23.4, 20.10, 필기 25.5, 21.3, 20.9

스키마(Schema)는 데이터베이스의 구조와 제약조건에 관한 전반적인 명세를 기술한 것이다.

종류	내용
23.4 외부 스키마	사용자나 응용 프로그램이 각 개인의 입장에서 필요로 하는 데이터베이스의 논리적 구조를 정의한 것
23.4, 필기 25.5, ... 개념 스키마	- 데이터베이스의 전체적인 논리적 구조 - 모든 응용 프로그램이나 사용자들이 필요로 하는 데이터를 종합한 조직 전체의 데이터베이스로, 하나만 존재함
23.4, 필기 20.9 내부 스키마	- 물리적 저장장치의 입장에서 본 데이터베이스 구조 - 실제로 저장될 레코드의 형식, 저장 데이터 항목의 표현 방법, 내부 레코드의 물리적 순서 등을 나타냄

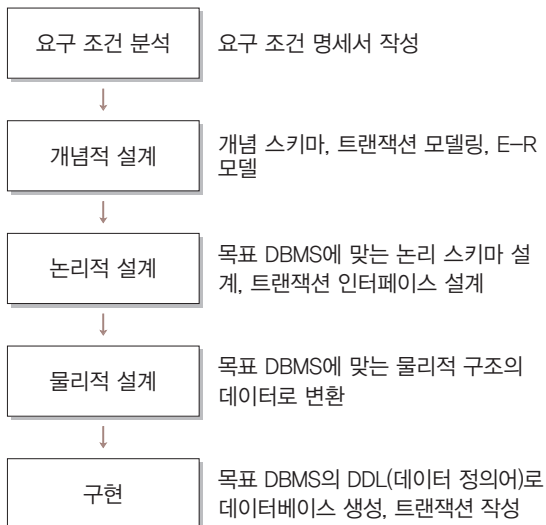
340081



035 데이터베이스 설계 순서

(A)

23.7, 20.7



340082



036 개념적 설계

(A)

21.4, 필기 23.2, 22.4

- 개념적 설계(정보 모델링, 개념화)는 정보의 구조를 얻기 위하여 현실 세계의 무한성과 계속성을 이해하고, 다른 사람과 통신하기 위하여 현실 세계에 대한 인식을 추상적 개념으로 표현하는 과정이다.
- 개념적 설계에서는 개념 스키마 모델링과 트랜잭션 모델링을 병행 수행한다.
- 개념적 설계에서는 요구 분석에서 나온 결과인 요구 조건 명세를 DBMS에 독립적인 E-R 다이어그램으로 작성한다.

340083



037 논리적 설계

(A)

21.4, 필기 24.7, 22.7, 20.6

- 논리적 설계(데이터 모델링)는 현실 세계에서 발생하는 자료를 컴퓨터가 이해하고 처리할 수 있는 물리적 저장장치에 저장할 수 있도록 변환하기 위해 특정 DBMS가 지원하는 논리적 자료 구조로 변환(mapping)시키는 과정이다.
- 개념 세계의 데이터를 필드로 기술된 데이터 타입과 이 데이터 타입들 간의 관계로 표현되는 논리적 구조의 데이터로 모델화한다.
- 개념적 설계가 개념 스키마를 설계하는 단계라면, 논리적 설계에서는 개념 스키마를 평가 및 정제하고 DBMS에 따라 서로 다른 논리적 스키마를 설계하는 단계이다.



038 물리적 설계

(A)

- 물리적 설계(데이터 구조화)는 논리적 설계에서 논리적 구조로 표현된 데이터를 디스크 등의 물리적 저장장치에 저장할 수 있는 물리적 구조의 데이터로 변환하는 과정이다.
- 물리적 설계에서는 다양한 데이터베이스 응용에 대해 처리 성능을 얻기 위해 데이터베이스 파일의 저장 구조 및 액세스 경로를 결정한다.
- 저장 레코드의 형식, 순서, 접근 경로, 조회 집중 레코드 등의 정보를 사용하여 데이터가 컴퓨터에 저장되는 방법을 묘사한다.



039 데이터 모델

(A)

- 데이터 모델은 현실 세계의 정보들을 컴퓨터에 표현하기 위해서 단순화, 추상화하여 체계적으로 표현한 개념적 모형이다.
- 데이터 모델에 표시할 요소

요소	내용
21.4, 필기 24.5, 23.2, 20.9 구조(Structure)	논리적으로 표현된 개체 타입들 간의 관계로서 데이터 구조 및 정적 성질 표현
21.4, 필기 25.8, 24.5, 23.2, ... 연산(Operation)	데이터베이스에 저장된 실제 데이터를 처리하는 작업에 대한 명세로서 데이터베이스를 조작하는 기본 도구
21.4, 필기 24.5, 23.2, 20.9 제약 조건(Constraint)	데이터베이스에 저장될 수 있는 실제 데이터의 논리적인 제약 조건



040 E-R 다이어그램

(B)

기호	기호 이름	의미
필기 25.5, 24.7, 24.5, 23.7, 22.7, ... 	사각형	개체(Entity) 타입
필기 24.7, 24.5, 23.7, 21.5, 21.3, ... 	마름모	관계(Relationship) 타입
필기 25.5, 24.7, 24.5, 23.7, 21.5, ... 	타원	속성(Attribute)
필기 22.3 	이중 타원	다중값 속성(복합 속성)
	밑줄 타원	기본키 속성
	복수 타원	복합 속성 예 성명은 성과 이름으로 구성
	관계	1:1, 1:N, N:M 등의 개체 간 관계에 대한 대응수를 선 위에 기술함
필기 24.7, 24.5, 21.5, 21.3, 20.9, ... 	선, 링크	개체 타입과 속성을 연결

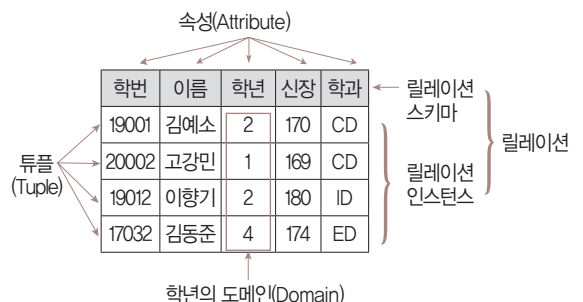


041 관계형 데이터베이스의 릴레이션 구조

(A)

릴레이션(Relation)은 데이터들을 표(Table)의 형태로 표현한 것으로, 구조를 나타내는 릴레이션 스키마와 실제 값들인 릴레이션 인스턴스로 구성된다.

〈학생〉 릴레이션



- 릴레이션 인스턴스 : 데이터 개체를 구성하고 있는 속성들에 데이터 타입이 정의되어 구체적인 데이터 값을 가진 것을 의미함

340071

25.11, 25.4, 24.7, 23.4, 21.4, 필기 25.5, 23.2, 22.4, 22.3, 21.3, 20.9, 20.8



042 튜플



- 튜플(Tuple)은 릴레이션을 구성하는 각각의 행을 말한다.
- 튜플은 속성의 모임으로 구성된다.
- 파일 구조에서 레코드와 같은 의미이다.
- 튜플의 수를 카디널리티(Cardinality) 또는 기수, 대응수라고 한다.

340092

25.7, 25.4, 24.7, 21.4, 필기 25.5, 23.2, 22.3, 21.5, 21.3, 20.9, 20.8



043 속성



- 속성(Attribute)은 데이터베이스를 구성하는 가장 작은 논리적 단위이다.
- 파일 구조상의 데이터 항목 또는 데이터 필드에 해당된다.
- 속성은 개체의 특성을 기술한다.
- 속성의 수를 디그리(Degree) 또는 차수라고 한다.

340093

25.4, 필기 25.8, 21.3, 20.6



044 도메인



- 도메인(Domain)은 하나의 애트리뷰트가 취할 수 있는 같은 타입의 원자(Atomic)값들의 집합이다.
- 도메인은 실제 애트리뷰트 값이 나타날 때 그 값의 합법 여부를 시스템이 검사하는 데에도 이용된다.
- 예 '성별' 애트리뷰트의 도메인은 "남"과 "여"로, 그 외의 값은 입력될 수 없다.

340094

24.10, 22.5, 필기 25.5, 22.7, 22.4, 20.6



045 후보키



- 후보키(Candidate Key)는 릴레이션을 구성하는 속성들 중에서 튜플을 유일하게 식별하기 위해 사용되는 속성들의 부분집합이다.
- 기본키로 사용할 수 있는 속성들을 말한다.
- 후보키는 유일성(Unique)과 최소성(Minimality)을 모두 만족시켜야 한다.

22.5, 필기 20.6

유일성 (Unique)

하나의 키 값으로 하나의 튜플만을 유일하게 식별할 수 있어야 함

22.5, 필기 20.6

최소성 (Minimality)

키를 구성하는 속성 하나를 제거하면 유일하게 식별할 수 없도록 꼭 필요한 최소의 속성으로 구성되어야 함

440076

24.10, 필기 22.7



046 대체키



- 대체키(Alternate Key)는 후보키가 둘 이상일 때 기본키를 제외한 나머지 후보키를 의미한다.
- 대체키를 보조키라고도 한다.

340097

24.10, 22.5, 필기 25.2, 24.5, 23.7, 22.7, 21.8, 20.9



047 슈퍼키



- 슈퍼키(Super Key)는 한 릴레이션 내에 있는 속성들의 집합으로 구성된 키를 말한다.
- 릴레이션을 구성하는 모든 튜플 중 슈퍼키로 구성된 속성의 집합과 동일한 값은 나타나지 않는다.
- 슈퍼키는 릴레이션을 구성하는 모든 튜플에 대해 유일성은 만족하지만, 최소성은 만족하지 못한다.

**048 외래키**

- 외래키(Foreign Key)는 다른 릴레이션의 기본키를 참조하는 속성 또는 속성들의 집합을 의미한다.
- 한 릴레이션에 속한 속성 A와 참조 릴레이션의 기본키인 B가 동일한 도메인 상에서 정의되었을 때의 속성 A를 외래키라고 한다.
- 외래키로 지정되면 참조 릴레이션의 기본키에 없는 값은 입력할 수 없다.

**049 무결성**

- 무결성(Integrity)은 데이터베이스에 저장된 데이터 값과 그것이 표현하는 현실 세계의 실제값이 일치하는 정확성을 의미한다.
- 개체 무결성: 기본 테이블의 기본키를 구성하는 어떤 속성도 Null 값이나 중복값을 가질 수 없다는 규정
- 참조 무결성: 외래키 값은 Null이거나 참조 릴레이션의 기본키 값과 동일해야 함. 즉 릴레이션은 참조할 수 없는 외래키 값을 가질 수 없다는 규정
- 도메인 무결성: 주어진 속성 값이 정의된 도메인에 속한 값이어야 한다는 규정

**050 관계대수**

- 관계대수는 관계형 데이터베이스에서 원하는 정보와 그 정보를 검색하기 위해서 어떻게 유도하는가를 기술하는 절차적인 언어이다.
- 관계대수는 릴레이션을 처리하기 위해 연산자와 연산규칙을 제공하며, 피연산자와 연산 결과가 모두 릴레이션이다.
- 관계대수는 질의에 대한 해를 구하기 위해 수행해야 할 연산의 순서를 명시한다.

**051 순수 관계 연산자**

23.10, 필기 25.5, 24.5, 23.7, 23.2, 21.3, 20.8

Select

- 릴레이션에 존재하는 튜플 중에서 선택 조건을 만족하는 튜플의 부분집합을 구하여 새로운 릴레이션을 만드는 연산이다.
- 릴레이션의 행에 해당하는 튜플(Tuple)을 구하는 것이므로 수평 연산이라고도 한다.
- 기호: σ (시그마)

25.7, 23.10, 22.10, 22.7, 필기 24.5, 23.7, 23.2, 21.8, 21.5, 20.8

Project

- 주어진 릴레이션에서 속성 리스트(Attribute List)에 제시된 속성 값만을 추출하여 새로운 릴레이션을 만드는 연산이다.
- 릴레이션의 열에 해당하는 속성을 추출하는 것이므로 수직 연산자라고도 한다.
- 기호: π (파이)

23.10, 22.10, 필기 21.8, 21.5, 20.8

Join

- 공통 속성을 중심으로 두 개의 릴레이션을 하나로 합쳐서 새로운 릴레이션을 만드는 연산이다.
- Join의 결과는 Cartesian Product(교차곱)를 수행한 다음 Select를 수행한 것과 같다.
- 기호: $\bowtie$

25.11, 23.10, 20.10, 필기 24.5, 21.8, 21.5, 20.8

Division

- $X \supset Y$ 인 두 개의 릴레이션 $R(X)$ 와 $S(Y)$ 가 있을 때, R 의 속성이 S 의 속성값을 모두 가진 튜플에서 S 가 가진 속성을 제외한 속성만을 구하는 연산이다.
- 기호: $\div$



052 일반 집합 연산자



22.10

합집합(UNION)

- 두 릴레이션에 존재하는 튜플의 합집합을 구하되, 결과로 생성된 릴레이션에서 중복되는 튜플은 제거되는 연산이다.
- 합집합의 카디널리티는 두 릴레이션 카디널리티의 합보다 크지 않다.
- 기호 : $\cup$

교집합(INTERSECTION)

- 두 릴레이션에 존재하는 튜플의 교집합을 구하는 연산이다.
- 교집합의 카디널리티는 두 릴레이션 중 카디널리티가 적은 릴레이션의 카디널리티보다 크지 않다.
- 기호 : $\cap$

22.10

차집합(DIFFERENCE)

- 두 릴레이션에 존재하는 튜플의 차집합을 구하는 연산이다.
- 차집합의 카디널리티는 릴레이션 R의 카디널리티보다 크지 않다.
- 기호 : $-$

22.10, 필기 24.5, 24.2, 21.8, 21.5

교차곱(CARTESIAN PRODUCT)

- 두 릴레이션에 있는 튜플들의 순서쌍을 구하는 연산이다.
- 교차곱의 디그리(Degree)는 두 릴레이션의 디그리를 더한 것과 같고, 카디널리티(Cardinality)는 두 릴레이션의 카디널리티를 곱한 것과 같다.
- 기호 : $\times$



053 관계해석



- 관계해석(Relational Calculus)은 관계 데이터의 연산을 표현하는 방법이다.
- 관계 데이터 모델의 제안자인 코드(E. F. Codd)가 수학의 Predicate Calculus(술어 해석)에 기반을 두고 관계 데이터베이스를 위해 제안했다.
- 관계해석은 원하는 정보가 무엇이라는 것만 정의하는 비절차적 특성을 지닌다.
- 원하는 정보를 정의할 때는 계산 수식을 사용한다.



054 이상



- 이상(Anomaly)이란 데이터베이스 내에 데이터들이 불필요하게 중복되어 릴레이션 조작 시 예기치 않게 발생하는 곤란한 현상을 의미한다.
- 삽입 이상(Insertion Anomaly) : 테이블에 데이터를 삽입할 때 의도와는 상관없이 원하지 않은 값들로 인해 삽입할 수 없게 되는 현상
- 삭제 이상(Deletion Anomaly) : 테이블에서 튜플을 삭제할 때 의도와는 상관없는 값들도 함께 삭제되는, 즉 연쇄 삭제가 발생하는 현상
- 갱신 이상(Update Anomaly) : 테이블에서 튜플에 있는 속성 값을 갱신할 때 일부 튜플의 정보만 갱신되어 정보에 불일치성(Inconsistency)이 생기는 현상



055 함수적 종속

(A)

필기 24.5, 24.2, 21.8

함수적 종속(Functional Dependency)

어떤 테이블 R에서 X와 Y를 각각 R의 속성 집합의 부분 집합이라 하자. 속성 X의 값 각각에 대해 시간에 관계없이 항상 속성 Y의 값이 오직 하나만 연관되어 있을 때 Y는 X에 함수적 종속 또는 X가 Y를 함수적으로 결정한다고 하고, $X \rightarrow Y$ 로 표기한다.

22.7

완전 함수적 종속(Full Functional Dependency)

어떤 테이블 R에서 속성 Y가 다른 속성 집합 X 전체에 대해 함수적 종속이면서 속성 집합 X의 어떠한 진부분 집합 Z(즉, $Z \subset X$)에도 함수적 종속이 아닐 때 속성 Y는 속성 집합 X에 완전 함수적 종속이라고 한다.

22.7

부분 함수적 종속(Partial Functional Dependency)

어떤 테이블 R에서 속성 Y가 다른 속성 집합 X 전체에 대해 함수적 종속이면서 속성 집합 X의 임의의 진부분 집합에 대해 함수적 종속일 때, 속성 Y는 속성 집합 X에 부분 함수적 종속이라고 한다.

22.7, 필기 24.2, 22.3, 20.8, 20.6

이행적 함수적 종속(Transitive Functional Dependency)

$X \rightarrow Y$ 이고 $Y \rightarrow Z$ 일 때 $X \rightarrow Z$ 를 만족하는 관계를 의미한다.



056 정규화

(C)

- 정규화(Normalization)는 테이블의 속성들이 상호 종속적인 관계를 갖는 특성을 이용하여 테이블을 무손실 분해하는 과정이다.
- 정규화의 목적은 가능한 한 중복을 제거하여 삽입, 삭제, 갱신 이상의 발생 가능성을 줄이는 것이다.
- 정규형에는 제 1정규형(1NF), 제 2정규형(2NF), 제 3정규형(3NF), BCNF, 제 4정규형(4NF), 제 5정규형(5NF)이 있으며, 순서대로 정규화의 정도가 높아진다.



057 정규화 과정



아래의 <주문목록> 테이블을 가지고 정규화 과정을 살펴보자. <주문목록> 테이블의 기본키(Primary Key)는 제품번호이다.

<주문목록>

제품번호	제품명	재고수량	주문번호	고객번호	주소	주문수량
1001	모니터	2000	A345	100	서울	150
			D347	200	부산	300
1007	마우스	9000	A210	300	광주	600
			A345	100	서울	400
			B230	200	부산	700
1201	키보드	2100	D347	200	부산	300

• 제 1정규형(1NF; First Normal Form)

- 제 1정규형은 테이블 R에 속한 모든 속성의 도메인(Domain)이 원자 값(Atomic Value)만으로 되어 있는 정규형이다. 즉 테이블의 모든 속성 값이 원자 값으로만 되어 있는 정규형이다.
- <주문목록> 테이블에서는 하나의 제품에 대해 여러 개의 주문 관련 정보(주문번호, 고객번호, 주소, 주문수량)가 발생하고 있다. 따라서 <주문목록>테이블은 제 1정규형이 아니다.

예제 1 <주문목록> 테이블에서 반복되는 주문 관련 정보를 분리하여 제 1정규형으로 만드시오.

<주문목록>

제품번호	제품명	재고수량	주문번호	고객번호	주소	주문수량
1001	모니터	2000	A345	100	서울	150
			D347	200	부산	300
1007	마우스	9000	A210	300	광주	600
			A345	100	서울	400
			B230	200	부산	700
1201	키보드	2100	D347	200	부산	300



<제품>

제품번호	제품명	재고수량
1001	모니터	2000
1007	마우스	9000
1201	키보드	2100

<제품주문>

주문번호	제품번호	고객번호	주소	주문수량
A345	1001	100	서울	150
D347	1001	200	부산	300
A210	1007	300	광주	600
A345	1007	100	서울	400
B230	1007	200	부산	700
D347	1201	200	부산	300

해설 <주문목록> 테이블에서 반복되는 주문 관련 정보인 주문번호, 고객번호, 주소, 주문수량을 분리하면 위와 같이 제 1정규형인 <제품> 테이블과 <제품주문> 테이블이 만들어진다.

- 1차 정규화 과정으로 생성된 <제품주문> 테이블의 기본키는 (주문번호, 제품번호)이고, 다음과 같은 함수적 종속이 존재한다.

주문번호, 제품번호 → 고객번호, 주소, 주문수량
 주문번호 → 고객번호, 주소
 고객번호 → 주소

• 제 2정규형(2NF; Second Normal Form)

- 제 2정규형은 테이블 R이 제 1정규형이고, 기본키가 아닌 모든 속성이 기본키에 대하여 완전 함수적 종속을 만족하는 정규형이다.
- <주문목록> 테이블이 <제품> 테이블과 <제품주문> 테이블로 무손실 분해되면서 모두 제 1정규형이 되었지만 그 중 <제품주문> 테이블에는 기본키인 (주문번호, 제품번호)에 완전 함수적 종속이 되지 않는 속성이 존재한다. 즉 주문수량은 기본키에 대해 완전 함수적 종속이지만 고객번호와 주소는 주문번호에 의해서도 결정될 수 있으므로, 기본키에 대해 완전 함수적 종속이 아니다. 따라서 <제품주문> 테이블은 제 2정규형이 아니다.

예제 2 <제품주문> 테이블에서 주문번호에 함수적 종속이 되는 속성들을 분리하여 제 2정규형을 만드시오.

<제품주문>

주문번호	제품번호	고객번호	주소	주문수량
A345	1001	100	서울	150
D347	1001	200	부산	300
A210	1007	300	광주	600
A345	1007	100	서울	400
B230	1007	200	부산	700
D347	1201	200	부산	300

<주문목록>

주문번호	제품번호	주문수량
A345	1001	150
D347	1001	300
A210	1007	600
A345	1007	400
B230	1007	700
D347	1201	300

<주문>

주문번호	고객번호	주소
A345	100	서울
D347	200	부산
A210	300	광주
B230	200	부산

해설 <제품주문> 테이블에서 주문번호에 함수적 종속이 되는 속성인 고객번호와 주소를 분리(즉 부분 함수적 종속을 제거)해 내면 위와 같이 제 2정규형인 <주문목록> 테이블과 <주문> 테이블로 무손실 분해된다.

- 제 2정규화 과정을 거쳐 생성된 <주문> 테이블의 기본키는 주문번호이다. 그리고 <주문> 테이블에는 아직도 다음과 같은 함수적 종속들이 존재한다.

주문번호 → 고객번호, 주소
 고객번호 → 주소

• 제 3정규형(3NF; Third Normal Form)

- 제 3정규형은 테이블 R이 제 2정규형이고 기본키가 아닌 모든 속성이 기본키에 대해 이행적 함수적 종속(Transitive Functional Dependency)을 만족하지 않는 정규형이다.
- <제품주문> 테이블이 <주문목록> 테이블과 <주문> 테이블로 무손실 분해되면서 모두 제 2정규형이 되었다. 그러나 <주문> 테이블에서 고객번호가 주문번호에 함수적 종속이고, 주소가 고객번호에 함수적 종속이므로 주소는 기본키인 주문번호에 대해 이행적 함수적 종속을 만족한다. 즉 주문번호 → 고객번호이고, 고객번호 → 주소이므로 주문번호 → 주소는 이행적 함수적 종속이 된다. 따라서 <주문> 테이블은 제 3정규형이 아니다.

예제 3 〈주문〉 테이블에서 이행적 함수적 종속을 제거하여 제 3정규형을 만드시오.

〈주문〉

주문번호	고객번호	주소
A345	100	서울
D347	200	부산
A210	300	광주
B230	200	부산



〈주문〉

주문번호	고객번호
A345	100
D347	200
A210	300
B230	200

〈고객〉

고객번호	주소
100	서울
200	부산
300	광주

해설 〈주문〉 테이블에서 이행적 함수적 종속(즉 주문번호 → 주소)을 제거하여 무손실 분해함으로써 위와 같이 제 3정규형인 〈주문〉 테이블과 〈고객〉 테이블이 생성된다.

• BCNF(Boyce–Codd Normal Form)

- BCNF는 테이블 R에서 모든 결정자가 후보키(Candidate Key)인 정규형이다.
- 일반적으로 제 3정규형에 후보키가 여러 개 존재하고, 이러한 후보키들이 서로 중첩되어 나타나는 경우에 적용 가능하다.
- 아래의 〈수강_교수〉 테이블(제 3정규형)은 함수적 종속{(학번, 과목명) → 담당교수, (학번, 담당교수) → 과목명, 담당교수 → 과목명}을 만족하고 있다. 〈수강_교수〉 테이블의 후보키는 (학번, 과목명)과 (학번, 담당교수)이다.

〈수강_교수〉

학번	과목명	담당교수
211746	데이터베이스	홍길동
211747	네트워크	유관순
211748	인공지능	윤봉길
211749	데이터베이스	홍길동
211747	데이터베이스	이순신
211749	네트워크	유관순

- 〈수강_교수〉 테이블에서 결정자 중 후보키가 아닌 속성이 존재한다. 즉 함수적 종속 담당교수 → 과목명이 존재하는데, 담당교수가 〈수강_교수〉 테이블에서 후보키가 아니기 때문에 〈수강_교수〉 테이블은 BCNF가 아니다.

예제 4 〈수강_교수〉 테이블에서 결정자가 후보키가 아닌 속성을 분리하여 BCNF를 만드시오.

〈수강〉

학번	담당교수
211746	홍길동
211747	유관순
211748	윤봉길
211749	홍길동
211747	이순신
211749	유관순

〈교수〉

담당교수	과목명
홍길동	데이터베이스
이순신	데이터베이스
윤봉길	인공지능
유관순	네트워크

해설 〈수강_교수〉 테이블에서 BCNF를 만족하지 못하게 하는 속성(즉 담당교수 → 과목명)을 분리해내면 위와 같이 BCNF인 〈수강〉 테이블과 〈교수〉 테이블로 무손실 분해된다.

• 제 4정규형(4NF; Fourth Normal Form)

제 4정규형은 테이블 R에 다중 값 종속(MVD; Multi Valued Dependency) $A \twoheadrightarrow B$ 가 존재할 경우 R의 모든 속성이 A에 함수적 종속 관계를 만족하는 정규형이다.

※ 다중 값 종속(다치 종속) : A, B, C 3개의 속성을 가진 테이블 R에서 어떤 복합 속성(A, C)에 대응하는 B 값의 집합이 A 값에만 종속되고 C 값에는 무관하면, B는 A에 다중 값 종속이라 하고, $A \twoheadrightarrow B$ 로 표기함

• 제 5정규형(5NF; Fifth Normal Form)

제 5정규형은 테이블 R의 모든 조인 종속(JD; Join Dependency)이 R의 후보키를 통해서만 성립되는 정규형이다.

※ 조인 종속 : 어떤 테이블 R의 속성에 대한 부분 집합 X, Y, ..., Z가 있다고 하자. 이때 만일 테이블 R이 자신의 프로젝션(Projection) X, Y, ..., Z를 모두 조인한 결과와 동일한 경우 테이블 R은 조인 종속 JD(X, Y, ..., Z)를 만족한다고 함

340108



필기 23.2, 21.5, 21.3, 20.9, 20.8, 20.6

058 정규화 과정 정리

(B)

비정규 릴레이션

↓ 도메인이 원자값

1NF

↓ 부분적 함수 종속 제거

2NF

↓ 이행적 함수 종속 제거

3NF

↓ 결정자이면서 후보키가 아닌 것 제거

BCNF

↓ 다치 종속 제거

4NF

↓ 조인 종속성 이용

5NF

정규화 단계 암기 요령

두부를 좋아하는 정규화가 두부가게에 가서 가게에 있는 두부를 다 달라고 말하니 주인이 깜짝 놀라며 말했다.

두부이겠다줘? ≡ 두부이겠다조

도메인이 원자값
부분적 함수 종속 제거
이행적 함수 종속 제거
결정자이면서 후보키가 아닌 것 제거
다치 종속 제거
조인 종속성 이용

340109



24.7, 21.4, 20.5, 필기 25.5, 24.2, 23.7, 20.9

059 반정규화

(A)

- 반정규화(Denormalization)는 시스템의 성능을 향상하고 개발 및 운영의 편의성 등을 높이기 위해 정규화된 데이터 모델을 의도적으로 통합, 중복, 분리하여 정규화 원칙을 위배하는 행위이다.
- 반정규화를 수행하면 시스템의 성능이 향상되고 관리 효율성은 증가하지만 데이터의 일관성 및 정합성이 저하될 수 있다.
- 과도한 반정규화는 오히려 성능을 저하시킬 수 있다.

340111



필기 25.8, 25.5, 24.7, 24.5, 24.2, 23.7, 22.7, 21.5, 21.3

060 시스템 카탈로그

(B)

- 시스템 카탈로그(System Catalog)는 시스템 그 자체에 관련이 있는 다양한 객체에 관한 정보를 포함하는 시스템 데이터베이스이다.
- 시스템 카탈로그 내의 각 테이블은 사용자를 포함하여 DBMS에서 지원하는 모든 데이터 객체에 대한 정의나 명세에 관한 정보를 유지 관리하는 시스템 테이블이다.
- 카탈로그들이 생성되면 데이터 사전(Data Dictionary)에 저장되기 때문에 좁은 의미로는 카탈로그를 데이터 사전이라고도 한다.

340113

21.7, 20.5, 필기 25.8, 25.5, 25.2, 24.7, 24.5, 24.2, 23.7, 23.5, 23.2, 22.7, ...



061 트랜잭션의 특성

A

특성	의미
21.7, 20.5, 필기 25.2, 24.5, ... Atomicity(원자성)	트랜잭션의 연산은 데이터베이스에 모두 반영되도록 완료(Commit)되든지 아니면 전혀 반영되지 않도록 복구(Rollback)되어야 함
20.5, 필기 25.8, 25.5, ... Consistency (일관성)	트랜잭션이 그 실행을 성공적으로 완료하면 언제나 일관성 있는 데이터베이스 상태로 변환함
20.5, 필기 25.5, 25.2, 23.7, ... Isolation (독립성, 격리성, 순차성)	둘 이상의 트랜잭션이 동시에 병행 실행되는 경우 어느 하나의 트랜잭션 실행중에 다른 트랜잭션의 연산이 끼어들 수 없음
20.5, 필기 25.5, 25.2, 24.7, ... Durability (영속성, 지속성)	성공적으로 완료된 트랜잭션의 결과는 시스템이 고장나더라도 영구적으로 반영되어야 함

340114

필기 22.7, 20.9



062 CRUD 분석

C

- CRUD 분석은 프로세스와 테이블 간에 CRUD 매트릭스를 만들어서 트랜잭션을 분석하는 것이다.
- CRUD 분석을 통해 많은 트랜잭션이 몰리는 테이블을 파악할 수 있으므로 디스크 구성 시 유용한 자료로 활용할 수 있다.
- CRUD 매트릭스의 각 셀에는 Create(생성), Read(읽기), Update(갱신), Delete(삭제)의 앞 글자가 들어간다.

340115

필기 24.7, 21.8, 21.3



063 인덱스

C

- 인덱스(Index, 색인)는 데이터 레코드를 빠르게 접근하기 위해 <키 값, 포인터> 쌍으로 구성되는 데이터 구조이다.
- 인덱스는 레코드가 저장된 물리적 구조에 접근하는 방법을 제공한다.
- 인덱스를 통해서 파일의 레코드에 빠르게 액세스 할 수 있다.

340117

필기 25.8, 25.5, 25.2, 24.7, 24.5, 24.2, 23.7, 22.4, 20.9, 20.6



064 뷰

B

- 뷰(View)는 사용자에게 접근이 허용된 자료만을 제한적으로 보여주기 위해 하나 이상의 기본 테이블로부터 유도된, 이름을 가지는 가상 테이블이다.
- 뷰가 정의된 기본 테이블이나 뷰를 삭제하면 그 테이블이나 뷰를 기초로 정의된 다른 뷰도 자동으로 삭제된다.
- 뷰를 정의할 때는 CREATE문, 제거할 때는 DROP 문을 사용한다.

340119

필기 25.8, 25.2, 24.7, 24.5, 24.2, 23.2, 22.7, 21.5, 20.8



065 파티션의 종류

B

필기 25.8, 24.7, 24.5, ... 범위 분할 (Range Partitioning)	지정한 열의 값을 기준으로 분할함 예 일별, 월별, 분기별 등
필기 25.8, 25.2, 24.7, ... 해시 분할 (Hash Partitioning)	- 해시 함수를 적용한 결과 값에 따라 데이터를 분할함 - 특정 파티션에 데이터가 집중되는 범위 분할의 단점을 보완한 것으로, 데이터를 고르게 분산할 때 유용함 - 특정 데이터가 어디에 있는지 판단할 수 없음 - 고객번호, 주민번호 등과 같이 데이터가 고르게 걸림에 효과적임
필기 25.8, 24.7, 20.8 조합 분할 (Composite Partitioning)	- 범위 분할로 분할한 다음 해시 함수를 적용하여 다시 분할하는 방식 - 범위 분할한 파티션이 너무 커서 관리가 어려울 때 유용함



066 분산 데이터베이스의 목표

(B)

- 위치 투명성(Location Transparency) : 액세스하려는 데이터베이스의 실제 위치를 알 필요 없이 단지 데이터베이스의 논리적인 명칭만으로 액세스할 수 있음
- 중복 투명성(Replication Transparency) : 동일 데이터가 여러 곳에 중복되어 있더라도 사용자는 마치 하나의 데이터만 존재하는 것처럼 사용하고, 시스템은 자동으로 여러 자료에 대한 작업을 수행함
- 병행 투명성(Concurrency Transparency) : 분산 데이터베이스와 관련된 다수의 트랜잭션들이 동시에 실행되더라도 그 트랜잭션의 결과는 영향을 받지 않음
- 장애 투명성(Failure Transparency) : 트랜잭션, DBMS, 네트워크, 컴퓨터 장애에도 불구하고 트랜잭션을 정확하게 처리함



067 RTO/RPO

(A)

20.7 RTO(Recovery Time Objective, 목표 복구 시간)	비상사태 또는 업무 중단 시점으로부터 복구되어 가동될 때까지의 소요 시간을 의미함 예) 장애 발생 후 6시간 내 복구 가능
RPO(Recovery Point Objective, 목표 복구 시점)	비상사태 또는 업무 중단 시점으로부터 데이터를 복구할 수 있는 기준점을 의미함 예) 장애 발생 전인 지난 주 금요일에 백업시켜 둔 복원 시점으로 복구 가능



068 임의 접근통제

(A)

- 임의 접근통제(DAC; Discretionary Access Control)는 데이터에 접근하는 사용자의 신원에 따라 접근 권한을 부여하는 방식이다.
- 데이터 소유자가 접근통제 권한을 지정하고 제어한다.
- 객체를 생성한 사용자가 생성된 객체에 대한 모든 권한을 부여받고, 부여된 권한을 다른 사용자에게 허가할 수도 있다.



069 강제 접근통제

(A)

- 강제 접근통제(MAC; Mandatory Access Control)는 주체와 객체의 등급을 비교하여 접근 권한을 부여하는 방식이다.
- 시스템이 접근통제 권한을 지정한다.
- 데이터베이스 객체별로 보안 등급을 부여할 수 있다.
- 사용자별로 인가 등급을 부여할 수 있다.



070 역할기반 접근통제

(A)

- 역할기반 접근통제(RBAC; Role Based Access Control)는 사용자의 역할에 따라 접근 권한을 부여하는 방식이다.
- 중앙관리자가 접근통제 권한을 지정한다.
- 임의 접근통제와 강제 접근통제의 단점을 보완하였다.
- 다중 프로그래밍 환경에 최적화된 방식이다.



071 DAS

(B)

- DAS(Direct Attached Storage)는 서버와 저장장치를 전용 케이블로 직접 연결하는 방식이다.
- 일반 가정에서 컴퓨터에 외장하드를 연결하는 것이 여기에 해당된다.
- 직접 연결 방식이므로 다른 서버에서 접근할 수 없고 파일을 공유할 수 없다.



072 SAN

C

- SAN(Storage Area Network)은 DAS의 빠른 처리와 NAS의 파일 공유 장점을 혼합한 방식으로, 서버와 저장장치를 연결하는 전용 네트워크를 별도로 구성하는 방식이다.
- 파이버 채널(Fibre Channel, 광 채널) 스위치를 이용하여 네트워크를 구성한다.
- 파이버 채널 스위치는 서버와 저장장치를 광케이블로 연결하므로 처리 속도가 빠르다.
- 서버들이 저장장치 및 파일을 공유할 수 있다.



073 자료 구조의 분류

B

- 선형 구조 : 배열, 선형 리스트(연속 리스트, 연결 리스트), 스택, 큐, 덱
- 비선형 구조 : 트리, 그래프



074 스택

B

- 스택(Stack)은 리스트의 한쪽 끝으로만 자료의 삽입, 삭제 작업이 이루어지는 자료구조이다.
- 후입선출(LIFO; Last In First Out) 방식으로 자료를 처리한다.
- 저장할 기억 공간이 없는 상태에서 데이터가 삽입되면 오버플로(Overflow)가 발생한다.
- 삭제할 데이터가 없는 상태에서 데이터를 삭제하면 언더플로(Underflow)가 발생한다.

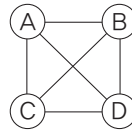


075 방향/무방향 그래프의 최대 간선 수

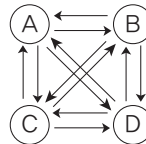
C

- 방향 그래프의 최대 간선 수 : $n(n-1)$
- 무방향 그래프에서 최대 간선 수 : $n(n-1)/2$
※ n 은 정점의 개수이다.

예제 정점이 4개인 경우 무방향 그래프와 방향 그래프의 최대 간선 수를 구하시오.



- 무방향 그래프의 최대 간선 수 : $4(4-1)/2 = 6$

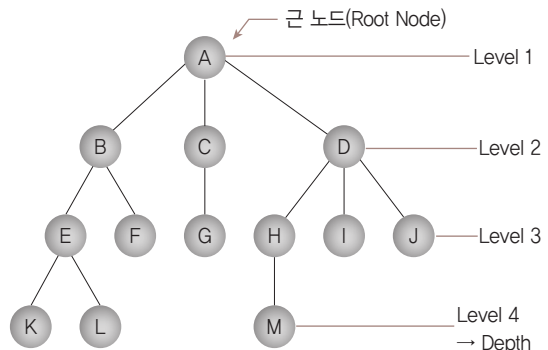


- 방향 그래프의 최대 간선 수 : $4(4-1) = 12$



076 트리 관련 용어

B



- 노드(Node) : 트리의 기본 요소로서 자료 항목과 다른 항목에 대한 가지(Branch)를 합친 것
예 A, B, C, D, E, F, G, H, I, J, K, L, M
- 근 노드(Root Node) : 트리의 맨 위에 있는 노드 예 A
- 디그리(Degree, 차수) : 각 노드에서 뻗어나온 가지의 수
예 A=3, B=2, C=1, D=3

- 단말 노드(Terminal Node) = 잎 노드(Leaf Node) : 자식이 하나도 없는 노드, 즉 Degree가 0인 노드

예 K, L, F, G, M, I, J

- Level : 근 노드의 Level을 1로 가정한 후 어떤 Level이 L이면 자식 노드는 L+1

예 H의 레벨은 3

- 깊이(Depth, Height) : Tree에서 노드가 가질 수 있는 최대의 레벨

예 위 트리의 깊이는 4

- 숲(Forest) : 여러 개의 트리가 모여 있는 것

예 위 트리에서 근 노드 A를 제거하면 B, C, D를 근 노드로 하는 세 개의 트리가 있는 숲이 생긴다.

- 트리의 디그리 : 노드들의 디그리 중에서 가장 많은 수

예 노드 A나 D가 세 개의 디그리를 가지므로 위 트리의 디그리는 3이다.

340138

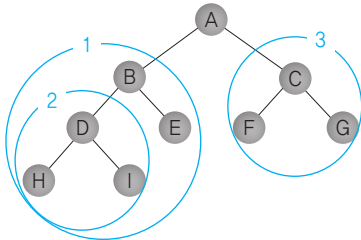


077 Preorder 운행법

(B)

Preorder 운행법은 이진 트리를 Root → Left → Right 순으로 운행하며 노드들을 찾아가는 방법이다.

Preorder 운행법의 방문 순서



※ 서브트리를 하나의 노드로 생각할 수 있도록 그림과 같이 서브트리 단위로 묶는다. 다른 운행법 모두 공통으로 사용한다.

- 1 Preorder는 Root → Left → Right이므로 A13이 된다.
- 2 1은 B2E이므로 AB2E3이 된다.
- 3 2는 DHI이므로 ABDHIE3이 된다.
- 4 3은 CFG이므로 ABDHIECFG가 된다.

• 방문 순서 : ABDHIECFG

340139

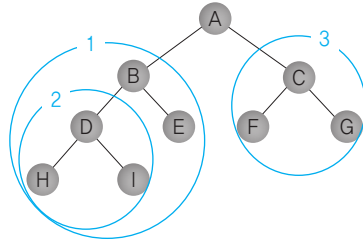


078 Inorder 운행법

(B)

Inorder 운행법은 이진 트리를 Left → Root → Right 순으로 운행하며 노드들을 찾아가는 방법이다.

Inorder 운행법의 방문 순서



- 1 Inorder는 Left → Root → Right이므로 1A3이 된다.
- 2 1은 2BE이므로 2BEA3이 된다.
- 3 2는 HDI이므로 HDIBE3이 된다.
- 4 3은 FCG이므로 HDIBEAFCG가 된다.

• 방문 순서 : HDIBEAFCG

440115

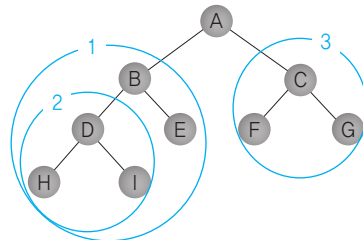


079 Postorder 운행법

(C)

Postorder 운행법은 이진 트리를 Left → Right → Root 순으로 운행하며 노드들을 찾아가는 방법이다.

Postorder 운행법의 방문 순서



- 1 Postorder는 Left → Right → Root이므로 13A가 된다.
- 2 1은 2EB이므로 2EB3A가 된다.
- 3 2는 HID이므로 HIDE3A가 된다.
- 4 3은 FGC이므로 HIDE3A가 된다.

• 방문 순서 : HIDE3A



080

Postfix로 표기된 수식을 Infix로 바꾸기

B

Postfix는 Infix 표기법에서 연산자를 해당 피연산자 두 개의 뒤로 이동한 것이므로 연산자를 다시 해당 피연산자 두 개의 가운데로 옮기면 된다.

$$ABC - / DEF + * + \rightarrow A / (B - C) + D * (E + F)$$

- ❶ 먼저 인접한 피연산자 두 개와 오른쪽의 연산자를 괄호로 묶는다.

$$((A(BC-)))(D(EF+)*)+$$

- ❷ 연산자를 해당 피연산자의 가운데로 이동시킨다.

$$((A(B C -) /))(D(E F +) *) +$$


$$((A/(B-C)))+(D*(E+F)))$$

- ❸ 필요 없는 괄호를 제거한다.

$$A/(B-C)+D*(E+F)$$


081

삽입 정렬

C

- 삽입 정렬(Insertion Sort)은 가장 간단한 정렬 방식으로, 이미 순서화된 파일에 새로운 하나의 레코드를 순서에 맞게 삽입시켜 정렬하는 방식이다.
- 평균과 최악 모두 수행 시간 복잡도는 $O(n^2)$ 이다.

예제 8, 5, 6, 2, 4를 삽입 정렬로 정렬하시오.

- 초기상태:

8	5	6	2	4
---	---	---	---	---

- 1회전:

8	5	6	2	4
---	---	---	---	---

 →

5	8	6	2	4
---	---	---	---	---

두 번째 값을 첫 번째 값과 비교하여 5를 첫 번째 자리에 삽입하고 8을 한 칸 뒤로 이동시킨다.

- 2회전:

5	8	6	2	4
---	---	---	---	---

 →

5	6	8	2	4
---	---	---	---	---

세 번째 값을 첫 번째, 두 번째 값과 비교하여 6을 8자리에 삽입하고 8은 한 칸 뒤로 이동시킨다.

- 3회전:

5	6	8	2	4
---	---	---	---	---

 →

2	5	6	8	4
---	---	---	---	---

네 번째 값 2를 처음부터 비교하여 맨 처음에 삽입하고 나머지를 한 칸씩 뒤로 이동시킨다.

- 4회전:

2	5	6	8	4
---	---	---	---	---

 →

2	4	5	6	8
---	---	---	---	---

다섯 번째 값 4를 처음부터 비교하여 5자리에 삽입하고 나머지를 한 칸씩 뒤로 이동시킨다.



082 선택 정렬

B

- 선택 정렬(Selection Sort)은 n 개의 레코드 중에서 최소값을 찾아 첫 번째 레코드 위치에 놓고, 나머지 $(n-1)$ 개 중에서 다시 최소값을 찾아 두 번째 레코드 위치에 놓는 방식을 반복하여 정렬하는 방식이다.
- 평균과 최악 모두 수행 시간 복잡도는 $O(n^2)$ 이다.

예제 8, 5, 6, 2, 4를 선택 정렬로 정렬하시오.

• 초기 상태:

8	5	6	2	4
---	---	---	---	---

• 1회전:

8	5	6	2	4
---	---	---	---	---

 →

8	5	6	2	4
---	---	---	---	---

 →

2	5	6	8	4
---	---	---	---	---

첫 번째부터 마지막 값 중 최소값 2를 찾아 첫 번째 값 8과 위치를 교환한다.

• 2회전:

2	5	6	8	4
---	---	---	---	---

 →

2	5	6	8	4
---	---	---	---	---

 →

2	4	6	8	5
---	---	---	---	---

두 번째부터 마지막 값 중 최소값 4를 찾아 두 번째 값 5와 위치를 교환한다.

• 3회전:

2	4	6	8	5
---	---	---	---	---

 →

2	4	6	8	5
---	---	---	---	---

 →

2	4	5	8	6
---	---	---	---	---

세 번째부터 마지막 값 중 최소값 5를 찾아 세 번째 값 6과 위치를 교환한다.

• 4회전:

2	4	5	8	6
---	---	---	---	---

 →

2	4	5	8	6
---	---	---	---	---

 →

2	4	5	6	8
---	---	---	---	---

네 번째부터 마지막 값 중 최소값 6을 찾아 네 번째 값 8과 위치를 교환한다.



083 버블 정렬

B

- 버블 정렬(Bubble Sort)은 주어진 파일에서 인접한 두 개의 레코드 키 값을 비교하여 그 크기에 따라 레코드 위치를 서로 교환하는 정렬 방식이다.
- 평균과 최악 모두 수행 시간 복잡도는 $O(n^2)$ 이다.

예제 8, 5, 6, 2, 4를 버블 정렬로 정렬하시오.

• 초기 상태:

8	5	6	2	4
---	---	---	---	---

• 1회전:

5	8	6	2	4
---	---	---	---	---

 →

5	6	8	2	4
---	---	---	---	---

 →

5	6	2	8	4
---	---	---	---	---

→

5	6	2	4	8
---	---	---	---	---

• 2회전:

5	6	2	4	8
---	---	---	---	---

 →

5	2	6	4	8
---	---	---	---	---

 →

5	2	4	6	8
---	---	---	---	---

• 3회전:

2	5	4	6	8
---	---	---	---	---

 →

2	4	5	6	8
---	---	---	---	---

• 4회전:

2	4	5	6	8
---	---	---	---	---



084 퀵 정렬

B

- 퀵 정렬(Quick Sort)은 키를 기준으로 작은 값은 왼쪽, 큰 값은 오른쪽 서브 파일에 분해시키는 과정을 반복하는 정렬 방식이다.
- 레코드의 많은 자료 이동을 없애고 하나의 파일을 부분적으로 나누어 가면서 정렬한다.
- 평균 수행 시간 복잡도는 $O(n \log_2 n)$ 이고, 최악의 수행 시간 복잡도는 $O(n^2)$ 이다.

3장 통합 구현



340153

필기 21.5



085 연계 서버 / 송·수신 시스템 (C)

- 연계 서버 : 데이터를 전송 형식에 맞게 변환하고 송·수신을 수행하는 등 송·수신과 관련된 모든 처리 수행
- 송신 시스템 : 인터페이스 테이블 또는 파일의 데이터를 전송 형식에 맞도록 변환 및 송신을 수행하는 시스템
- 수신 시스템 : 수신 데이터를 인터페이스 테이블이나 파일로 생성하는 시스템

340156

20.5



086 XML (A)

- XML(eXtensible Markup Language)은 특수한 목적을 갖는 마크업 언어를 만드는 데 사용되는 다목적 마크업 언어이다.
- 웹브라우저 간 HTML 문법이 호환되지 않는 문제와 SGML의 복잡함을 해결하기 위하여 개발되었다.
- 사용자가 직접 문서의 태그(Tag)를 정의할 수 있으며, 다른 사용자가 정의한 태그를 사용할 수 있다.
- 트리 구조로 구성되어 있어 상위 태그는 여러 개의 하위 태그를 가질 수 있다.

340157

20.7



087 SOAP (A)

- SOAP(Simple Object Access Protocol)는 컴퓨터 네트워크 상에서 HTTP/HTTPS, SMTP 등을 이용하여 XML을 교환하기 위한 통신 규약이다.
- 웹 서비스에서 사용되는 메시지의 형식과 처리 방법을 지정한다.
- 기본적으로 HTTP 기반에서 동작하기 때문에 프록시와 방화벽의 영향 없이 통신할 수 있다.
- 최근에는 무거운 구조의 SOAP 대신 RESTful 프로토콜을 이용하기도 한다.

340158

21.4



088 WSDL (A)

- WSDL(Web Services Description Language)은 웹 서비스와 관련된 서식이나 프로토콜 등을 표준적인 방법으로 기술하고 게시하기 위한 언어이다.
- XML로 작성되며, UDDI의 기초가 된다.
- SOAP, XML 스키마와 결합하여 인터넷에서 웹 서비스를 제공하기 위해 사용된다.
- 클라이언트는 WSDL 파일을 읽어 서버에서 어떠한 조작이 가능한지를 파악할 수 있다.

4장 서버 프로그램 구현



340165

20.5



089 모듈화

(A)

- 모듈화(Modularity)는 소프트웨어의 성능 향상, 시스템의 수정 및 재사용, 유지 관리 등이 용이하도록 시스템의 기능들을 모듈 단위로 나누는 것을 의미한다.
- 모듈화는 모듈 간 결합도(Coupling)의 최소화와 모듈 내 요소들의 응집도(Cohesion)를 최대화하는 것이 목표이다.

340166

필기 24.7, 21.8



090 추상화

(C)

- 추상화(Abstraction)는 문제의 전체적이고 포괄적인 개념을 설계한 후 차례로 세분화하여 구체화시키 나가는 것이다.
- 추상화의 유형

과정 추상화	자세한 수행 과정을 정의하지 않고, 전반적인 흐름만 파악할 수 있게 설계하는 방법
자료 추상화	데이터의 세부적인 속성이나 용도를 정의하지 않고, 데이터 구조를 대표할 수 있는 표현으로 대체하는 방법
제어 추상화	이벤트 발생의 정확한 절차나 방법을 정의하지 않고, 대표할 수 있는 표현으로 대체하는 방법

340168

필기 25.5, 24.5, 21.8, 21.5



091 정보 은닉

(C)

- 정보 은닉(Information Hiding)은 한 모듈 내부에 포함된 절차와 자료들의 정보가 감추어져 다른 모듈이 접근하거나 변경하지 못하도록 하는 기법이다.
- 정보 은닉을 통해 모듈을 독립적으로 수행할 수 있다.
- 하나의 모듈이 변경되더라도 다른 모듈에 영향을 주지 않으므로 수정, 시험, 유지보수가 용이하다.

340171

필기 23.2, 20.8



092 협약(Contract)에 의한 설계

(C)

- 협약에 의한 설계는 컴포넌트를 설계할 때 클래스에 대한 여러 가정을 공유할 수 있도록 명세한 것이다.
- 컴포넌트에 대한 정확한 인터페이스를 명세한다.
- 명세에 포함될 조건

선행 조건 (Precondition)	오퍼레이션이 호출되기 전에 참이 되어야 할 조건
결과 조건 (Postcondition)	오퍼레이션이 수행된 후 만족되어야 할 조건
불변 조건 (Invariant)	오퍼레이션이 실행되는 동안 항상 만족되어야 할 조건

340175

필기 25.8, 24.7, 23.7, 23.2, 22.7, 21.8, 21.5, 20.9



093 파이프-필터 패턴

(B)

- 파이프-필터 패턴(Pipe-Filter Pattern)은 데이터 스트림 절차의 각 단계를 필터로 캡슐화하여 파이프를 통해 전송하는 패턴이다.
- 앞 시스템의 처리 결과물을 파이프를 통해 전달받아 처리한 후 그 결과물을 다시 파이프를 통해 다음 시스템으로 넘겨주는 패턴을 반복한다.
- 데이터 변환, 버퍼링, 동기화 등에 주로 사용된다.
- 대표적으로 UNIX의 셸(Shell)이 있다.

340177

필기 24.5, 23.5, 23.2, 21.8



094 기타 패턴

(B)

필기 24.5, 23.5, 23.2, 21.8

마스터-슬레이브 패턴(Master-Slave Pattern)

슬레이브 컴포넌트에서 처리된 결과물을 다시 돌려받는 방식으로 작업을 수행하는 패턴이다.

예 장애 허용 시스템, 병렬 컴퓨팅 시스템

브로커 패턴(Broker Pattern)

사용자가 원하는 서비스와 특성을 브로커 컴포넌트에 요청하면 브로커 컴포넌트가 요청에 맞는 컴포넌트와 사용자를 연결해주는 패턴이다.

예 분산 환경 시스템

피어-투-피어 패턴(Peer-To-Peer Pattern)

피어(Peer)라 불리는 하나의 컴포넌트가 클라이언트가 될수도, 서버가 될 수도 있는 패턴이다.

예 파일 공유 네트워크

이벤트-버스 패턴(Event-Bus Pattern)

소스가 특정 채널에 이벤트 메시지를 발행(Publish)하면, 해당 채널을 구독(Subscribe)한 리스너(Listener)들이 메시지를 받아 이벤트를 처리하는 패턴이다.

예 알림 서비스

블랙보드 패턴(Blackboard Pattern)

모든 컴포넌트들이 공유 데이터 저장소와 블랙보드 컴포넌트에 접근이 가능한 패턴이다.

예 음성 인식, 차량 식별, 신호 해석

필기 21.8

인터프리터 패턴(Interpreter Pattern)

프로그램 코드의 각 라인을 수행하는 방법을 지정하고, 기호마다 클래스를 갖도록 구성된 패턴이다.

예 번역기, 컴파일러, 인터프리터

340180



필기 25.8, 25.5, 25.2, 24.2, 23.5, 22.3, 21.8, 21.5, 20.8, 20.6

095 클래스



- 클래스(Class)는 공통된 속성과 연산을 갖는 객체의 집합이다.
- 각각의 객체들이 갖는 속성과 연산을 정의하고 있는 틀이다.
- 클래스에 속한 각각의 객체를 인스턴스(Instance)라고 한다.

440149



필기 25.2, 23.2, 22.7, 21.5

096 메시지



- 메시지(Message)는 객체들 간의 상호작용에 사용되는 수단으로, 객체의 동작이나 연산을 일으키는 외부의 요구 사항이다.
- 메시지를 받은 객체는 대응하는 연산을 수행하여 예상된 결과를 반환한다.

340182



필기 24.5, 24.2, 23.5, 21.8, 21.3, 20.9, 20.8

097 캡슐화



- 캡슐화(Encapsulation)는 외부에서의 접근을 제한하기 위해 인터페이스를 제외한 세부 내용을 은닉하는 것이다.
- 캡슐화된 객체는 외부 모듈의 변경으로 인한 파급 효과가 적다.
- 객체들 간에 메시지를 주고받을 때 상대 객체의 세부 내용은 알 필요가 없으므로 인터페이스가 단순해지고, 객체 간의 결합도가 낮아진다.

340186



필기 23.7, 21.8, 21.3

098 객체지향 분석



- 객체지향 분석(OOA; Object Oriented Analysis)은 사용자의 요구사항과 관련된 객체, 속성, 연산, 관계 등을 정의하여 모델링하는 작업이다.
- 개발을 위한 업무를 객체와 속성, 클래스와 멤버, 전체와 부분 등으로 나누어서 분석한다.
- 클래스를 식별하는 것이 객체지향 분석의 주요 목적이다.



099 객체지향 분석의 방법론

(C)

- Rumbaugh(럼바우) 방법 : 분석 활동을 객체 모델, 동적 모델, 기능 모델로 나누어 수행함
- Booch(부치) 방법 : 미시적(Micro) 개발 프로세스와 거시적(Macro) 개발 프로세스를 모두 사용하며, 클래스와 객체들을 분석 및 식별하고 클래스의 속성과 연산을 정의함
- Jacobson 방법 : 유스케이스(Use Case)를 강조하여 사용함
- Coad와 Yourdon 방법 : E-R 다이어그램을 사용하여 객체의 행위를 모델링하며, 객체 식별, 구조 식별, 주제 정의, 속성과 인스턴스 연결 정의, 연산과 메시지 연결 정의 등의 과정으로 구성함
- Wirfs-Brock 방법 : 분석과 설계 간의 구분이 없고, 고객 명세서를 평가해서 설계 작업까지 연속적으로 수행함



100 럼바우의 분석 기법

(A)

- 럼바우(Rumbaugh)의 분석 기법은 모든 소프트웨어 구성 요소를 그래픽 표기법을 이용하여 모델링하는 기법이다.
- 객체 모델링 기법(OMT, Object-Modeling Technique)이라고도 한다.
- 분석 활동은 '객체 모델링 → 동적 모델링 → 기능 모델링' 순으로 이루어 진다.
 - 객체 모델링(Object Modeling) : 정보 모델링(Information Modeling)이라고도 하며, 시스템에서 요구되는 객체를 찾아내어 속성과 연산 식별 및 객체들 간의 관계를 규정하여 객체 다이어그램으로 표시하는 모델링
 - 동적 모델링(Dynamic Modeling) : 상태 다이어그램을 이용하여 시간의 흐름에 따른 객체들 간의 제어 흐름, 상호 작용, 동작 순서 등의 동적인 행위를 표현하는 모델링
 - 기능 모델링(Functional Modeling) : 자료 흐름도(DFD)를 이용하여 다수의 프로세스들 간의 자료 흐름을 중심으로 처리 과정을 표현한 모델링



101 객체지향 설계 원칙(SOLID 원칙)

(A)

- 단일 책임 원칙(SRP) : 객체는 단 하나의 책임만 가져야 한다는 원칙
- 개방-폐쇄 원칙(OCP) : 기존의 코드를 변경하지 않고 기능을 추가할 수 있도록 설계해야 한다는 원칙
- 리스코프 치환 원칙(LSP) : 자식 클래스는 최소한 부모 클래스의 기능은 수행할 수 있어야 한다는 원칙
- 인터페이스 분리 원칙(ISP) : 자신이 사용하지 않는 인터페이스와 의존 관계를 맺거나 영향을 받지 않아야 한다는 원칙
- 의존 역전 원칙(DIP) : 의존 관계 성립 시 추상성이 높은 클래스와 의존 관계를 맺어야 한다는 원칙



102 모듈

(B)

- 모듈(Module)은 모듈화를 통해 분리된 시스템의 각 기능으로, 서브루틴, 서브시스템, 소프트웨어 내의 프로그램, 작업 단위 등을 의미한다.
- 모듈의 기능적 독립성은 소프트웨어를 구성하는 각 모듈의 기능이 서로 독립됨을 의미한다.
- 하나 또는 몇 개의 논리적인 기능을 수행하기 위한 명령어들의 집합이라고도 할 수 있다.
- 모듈의 독립성은 결합도(Coupling)와 응집도(Cohesion)에 의해 측정된다.

340191

필기 25.2, 24.5, 23.2, 21.8, 21.5, 21.3, 20.8, 20.6



103 결합도

(B)

- 결합도(Coupling)는 모듈 간에 상호 의존하는 정도 또는 두 모듈 사이의 연관 관계이다.
- 결합도가 약할수록 품질이 높고, 강할수록 품질이 낮다.
- 결합도의 종류와 강도

내용 결합도	공통 결합도	외부 결합도	제어 결합도	스탬프 결합도	자료 결합도
--------	--------	--------	--------	---------	--------

결합도 강함 ← → 결합도 약함

340192

25.4, 24.7, 21.10, 21.4, 필기 25.8, 23.7, 23.2, 22.7, 22.4, 20.9, 20.8



104 결합도의 종류

(A)

25.4, 21.4, 필기 25.8, 23.7, 23.2, 22.7, 22.4, 20.9

내용 결합도(Content Coupling)

한 모듈이 다른 모듈의 내부 기능 및 그 내부 자료를 직접 참조하거나 수정할 때의 결합도이다.

25.4, 21.4, 필기 23.7, 23.2, 20.9

공통(공유) 결합도(Common Coupling)

- 공유되는 공통 데이터 영역을 여러 모듈이 사용할 때의 결합도이다.
- 파라미터가 아닌 모듈 밖에 선언된 전역 변수를 사용하여 전역 변수를 갱신하는 방식으로 상호작용하는 때의 결합도이다.

필기 25.8, 22.7

외부 결합도(External Coupling)

어떤 모듈에서 선언한 데이터(변수)를 외부의 다른 모듈에서 참조할 때의 결합도이다.

24.7, 21.10, 필기 20.8

제어 결합도(Control Coupling)

- 어떤 모듈이 다른 모듈 내부의 논리적인 흐름을 제어하기 위해 제어 신호나 제어 요소를 전달하는 결합도이다.
- 하위 모듈에서 상위 모듈로 제어 신호가 이동하여 하위 모듈이 상위 모듈에게 처리 명령을 내리는 권리 전도 현상이 발생하게 된다.

25.4, 21.4, 필기 25.8, 23.2, 22.7, 22.4, 20.9

스탬프(검인) 결합도(Stamp Coupling)

모듈 간의 인터페이스로 배열이나 레코드 등의 자료 구조가 전달될 때의 결합도이다.

필기 23.7, 23.2, 22.7, 20.9

자료 결합도(Data Coupling)

모듈 간의 인터페이스가 자료 요소로만 구성될 때의 결합도이다.

340193

24.4, 필기 25.8, 24.5, 23.5, 23.2, 22.4, 21.5, 21.3, 20.6



105 응집도

(A)

- 응집도(Cohesion)는 모듈의 내부 요소들이 서로 관련되어 있는 정도이다.
- 응집도가 강할수록 품질이 높고, 약할수록 품질이 낮다.
- 응집도의 종류와 강도

기능적 응집도	순차적 응집도	교환적 응집도	절차적 응집도	시간적 응집도	논리적 응집도	우연적 응집도
---------	---------	---------	---------	---------	---------	---------

응집도 강함 ← → 응집도 약함

340194

24.7, 21.7, 필기 21.8, 20.9, 20.8



106 응집도의 종류

(A)

21.7

기능적 응집도(Functional Cohesion)

모듈 내부의 모든 기능 요소들이 단일 문제와 연관되어 수행될 경우의 응집도이다.

24.7

순차적 응집도(Sequential Cohesion)

모듈 내 하나의 활동으로부터 나온 출력 데이터를 그 다음 활동의 입력 데이터로 사용할 경우의 응집도이다.

21.7

교환(통신)적 응집도(Communication Cohesion)

동일한 입력과 출력을 사용하여 서로 다른 기능을 수행하는 구성 요소들이 모였을 경우의 응집도이다.

절차적 응집도(Procedural Cohesion)

모듈이 다수의 관련 기능을 가질 때 모듈 안의 구성 요소들이 그 기능을 순차적으로 수행할 경우의 응집도이다.

시간적 응집도(Temporal Cohesion)

특정 시간에 처리되는 몇 개의 기능을 모아 하나의 모듈로 작성할 경우의 응집도이다.

논리적 응집도(Logical Cohesion)

유사한 성격을 갖거나 특정 형태로 분류되는 처리 요소들로 하나의 모듈이 형성되는 경우의 응집도이다.

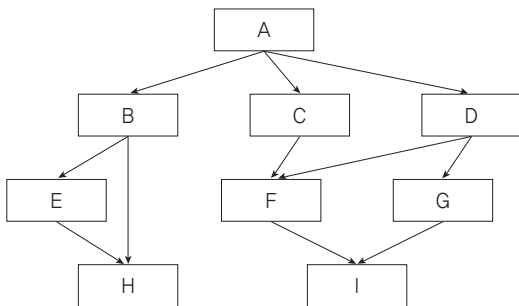
우연적 응집도(Coincidental Cohesion)

모듈 내부의 각 구성 요소들이 서로 관련 없는 요소로만 구성된 경우의 응집도이다.

**107 팬인 / 팬아웃**

- 팬인(Fan-In)은 어떤 모듈을 제어하는 모듈의 수이다.
- 팬아웃(Fan-Out)은 어떤 모듈에 의해 제어되는 모듈의 수를 의미한다.

예제 다음의 시스템 구조도에서 각 모듈의 팬인(Fan-In)과 팬아웃(Fan-Out)을 구하시오.

**해설**

- 팬인(Fan-In) : A는 0, B · C · D · E · G는 1, F · H · I는 2
- 팬아웃(Fan-Out) : H · I는 0, C · E · F · G는 1, B · D는 2, A는 3

**108 N-S 차트**

- N-S 차트(Nassi-Schneiderman Chart)는 논리의 기술에 중점을 두고 도형을 이용해 표현하는 방법이다.
- GOTO나 화살표를 사용하지 않는다.
- 연속, 선택 및 다중 선택, 반복의 3가지 제어 논리 구조로 표현한다.
- 조건이 복합되어 있는 곳의 처리를 시각적으로 명확히 식별하는 데 적합하다.

**109 IPC**

- IPC(Inter-Process Communication)는 모듈 간 통신 방식을 구현하기 위해 사용되는 대표적인 프로그래밍 인터페이스 집합이다.
- 복수의 프로세스를 수행하며 이뤄지는 프로세스 간 통신까지 구현이 가능하다.
- IPC의 대표 메소드 5가지
 - 공유 메모리(Shared Memory)
 - 소켓(Socket)
 - 세마포어(Semaphores)
 - 파이프와 네임드 파이프(Pipes & named Pipes)
 - 메시지 큐잉(Message Queueing)



110 테스트 케이스



- 테스트 케이스(Test Case)는 구현된 소프트웨어가 사용자의 요구사항을 정확하게 준수했는지를 확인하기 위한 테스트 항목에 대한 명세서이다.
- ISO/IEC/IEEE 29119-3 표준에 따른 테스트 케이스의 구성 요소
 - 식별자 : 항목 식별자, 일련번호
 - 테스트 항목 : 테스트 대상(모듈 또는 기능)
 - 입력 명세 : 테스트 데이터 또는 테스트 조건
 - 출력 명세 : 테스트 케이스 수행 시 예상되는 출력 결과
 - 환경 설정 : 필요한 하드웨어나 소프트웨어의 환경
 - 특수 절차 요구 : 테스트 케이스 수행 시 특별히 요구되는 절차
 - 의존성 기술 : 테스트 케이스 간의 의존성



111 재사용



- 재사용(Reuse)은 이미 개발된 기능들을 새로운 시스템이나 기능 개발에 사용하기 적합하도록 최적화하는 작업이다.
- 새로 개발하는데 필요한 비용과 시간을 절약할 수 있다.
- 누구나 이해할 수 있고 사용이 가능하도록 사용법을 공개해야 한다.
- 재사용 규모에 따른 분류

필기 20.9 함수와 객체	클래스나 메소드 단위의 소스 코드를 재사용함
필기 20.9 컴포넌트	컴포넌트 자체에 대한 수정 없이 인터페이스를 통해 통신하는 방식으로 재사용함
필기 20.9 애플리케이션	공통된 기능들을 제공하는 애플리케이션을 공유하는 방식으로 재사용함



112 코드의 종류



필기 23.7, 23.2, 20.6

순차 코드(Sequence Code)

자료의 발생 순서, 크기 순서 등 일정 기준에 따라서 최초의 자료부터 차례로 일련번호를 부여하는 방법으로, 순서 코드 또는 일련번호 코드라고도 한다.

예 1, 2, 3, 4, ...

블록 코드(Block Code)

코드화 대상 항목 중에서 공통성이 있는 것끼리 블록으로 구분하고, 각 블록 내에서 일련번호를 부여하는 방법으로, 구분 코드라고도 한다.

예 1001~1100 : 총무부, 1101~1200 : 영업부

10진 코드(Decimal Code)

코드화 대상 항목을 0~9까지 10진 분할하고, 다시 각각에 대하여 10진 분할하는 방법을 필요한 만큼 반복하는 방법으로, 도서 분류식 코드라고도 한다.

예 1000 : 공학, 1100 : 소프트웨어 공학, 1110 : 소프트웨어 설계

그룹 분류 코드(Group Classification Code)

코드화 대상 항목을 일정 기준에 따라 대분류, 중분류, 소분류 등으로 구분하고, 각 그룹 안에서 일련번호를 부여하는 방법이다.

예 1-01-001 : 본사-총무부-인사계, 2-01-001 : 지사-총무부-인사계

연상 코드(Mnemonic Code)

코드화 대상 항목의 명칭이나 약호와 관계있는 숫자나 문자, 기호를 이용하여 코드를 부여하는 방법이다.

예 TV-40 : 40인치 TV, L-15-220 : 15W 220V의 램프

필기 23.2, 20.9

표의 숫자 코드(Significant Digit Code)

코드화 대상 항목의 성질, 즉 길이, 넓이, 부피, 지름, 높이 등의 물리적 수치를 그대로 코드에 적용시키는 방법으로, 유효 숫자 코드라고도 한다.

예 120-720-1500 : 두께×폭×길이가 120×720×1500인 강판

합성 코드(Combined Code)

필요한 기능을 하나의 코드로 수행하기 어려운 경우 2개 이상의 코드를 조합하여 만드는 방법이다.

예 연상 코드 + 순차 코드

KE-711 : 대한항공 711기, AC-253 : 에어캐나다 253기

340204



20.11, 필기 25.8, 24.7, 23.5, 22.3, 21.8, 20.9, 20.8

113 디자인 패턴



- 디자인 패턴(Design Pattern)은 모듈 간의 관계 및 인터페이스를 설계할 때 참조할 수 있는 전형적인 해결 방식 또는 예제를 의미한다.
- 문제 및 배경, 실제 적용된 사례, 재사용이 가능한 샘플 코드 등으로 구성되어 있다.
- GOF의 디자인 패턴은 생성 패턴, 구조 패턴, 행위 패턴으로 구분된다.

340205



24.4, 21.10, 필기 25.8, 25.2, 24.5, 24.2, 23.7, 23.5, 23.2, 22.7, 22.3, 21.8, ...

114 생성 패턴



생성 패턴(Creational Pattern)은 클래스나 객체의 생성과 참조 과정을 정의하는 패턴이다.

24.4, 필기 24.5, 22.3, 21.3

추상 팩토리(Abstract Factory)

- 구체적인 클래스에 의존하지 않고, 인터페이스를 통해 서로 연관·의존하는 객체들의 그룹으로 생성하여 추상적으로 표현하는 패턴이다.
- 연관된 서브 클래스를 묶어 한 번에 교체하는 것이 가능하다.

필기 21.3

빌더(Builder)

- 작게 분리된 인스턴스를 건축 하듯이 조합하여 객체를 생성하는 패턴이다.

- 객체의 생성 과정과 표현 방법을 분리하고 있어, 동일한 객체 생성에서도 서로 다른 결과를 만들어 낼 수 있다.

21.10, 필기 25.5, 23.7, 23.2, 21.5, 20.8

팩토리 메소드(Factory Method)

- 객체 생성을 서브 클래스에서 처리하도록 분리하여 캡슐화한 패턴이다.
- 상위 클래스에서 인터페이스만 정의하고 실제 생성은 서브 클래스가 담당한다.
- 가상 생성자(Virtual Constructor) 패턴이라고도 한다.

필기 23.2, 21.5, 20.8

프로토타입(Prototype)

- 원본 객체를 복제하는 방법으로 객체를 생성하는 패턴이다.
- 일반적인 방법으로 객체를 생성하며, 비용이 큰 경우 주로 이용한다.

필기 25.2, 24.2, 23.5, 22.7, 21.8, 21.5, 21.3

싱글톤(Singleton)

- 하나의 객체를 생성하면 생성된 객체를 어디서든 참조할 수 있지만, 여러 프로세스가 동시에 참조할 수는 없는 패턴이다.
- 클래스 내에서 인스턴스가 하나뿐임을 보장하며, 불필요한 메모리 낭비를 최소화 할 수 있다.

440174



25.7, 25.4, 23.4, 22.10, 필기 25.8, 25.5, 23.2, 22.4, 21.5

115 구조 패턴



구조 패턴(Structural Pattern)은 구조가 복잡한 시스템을 개발하기 쉽도록 클래스나 객체들을 조합하여 더 큰 구조로 만드는 패턴이다.

25.4, 필기 25.8, 25.5, 23.2, 22.4, 21.5

어댑터(Adapter)

- 호환성이 없는 클래스들의 인터페이스를 다른 클래스가 이용할 수 있도록 변환해주는 패턴이다.
- 기존의 클래스를 이용하고 싶지만 인터페이스가 일치하지 않을 때 이용한다.

브리지(Bridge)

- 구현부에서 추상층을 분리하여 서로가 독립적으로 확장할 수 있도록 구성한 패턴이다.
- 기능과 구현을 두 개의 별도 클래스로 구현한다.

필기 25.8, 23.2

컴포지트(Composite)

- 여러 객체를 가진 복합 객체와 단일 객체를 구분 없이 다루고자 할 때 사용하는 패턴이다.
- 객체들을 트리 구조로 구성하여 디렉터리 안에 디렉터리가 있듯이 복합 객체 안에 복합 객체가 포함되는 구조를 구현할 수 있다.

필기 25.5, 23.2

데코레이터(Decorator)

- 객체 간의 결합을 통해 능동적으로 기능들을 확장할 수 있는 패턴이다.
- 임의의 객체에 부가적인 기능을 추가하기 위해 다른 객체들을 덧붙이는 방식으로 구현한다.

퍼싸드(Facade)

- 복잡한 서브 클래스들을 피해 더 상위 인터페이스를 구성함으로써 서브 클래스들의 기능을 간편하게 사용할 수 있도록 하는 패턴이다.
- 서브 클래스들 사이의 통합 인터페이스를 제공하는 Wrapper 객체가 필요하다.

플라이웨이트(Flyweight)

- 인스턴스가 필요할 때마다 매번 생성하는 것이 아니고 가능한 한 공유해서 사용함으로써 메모리를 절약하는 패턴이다.
- 다수의 유사 객체를 생성하거나 조작할 때 유용하게 사용할 수 있다.

25.7, 23.4, 필기 25.5, 22.4

프록시(Proxy)

- 복잡한 시스템을 개발하기 쉽도록 클래스나 객체들을 조합하는 패턴으로, 대리자라고도 불린다.
- 내부에서는 객체 간의 복잡한 관계를 단순하게 정리해 주고, 외부에서는 객체의 세부적인 내용을 숨겨주는 역할을 수행한다.

340207



24.10, 24.7, 22.10, 21.7, 20.7, 필기 25.5, 24.2, 23.5, 23.2, 21.8, 21.5, 20.8, 20.6

116 행위 패턴

행위 패턴(Behavioral Pattern)은 클래스나 객체들이 서로 상호작용하는 방법이나 책임 분배 방법을 정의하는 패턴이다.

책임 연쇄(Chain of Responsibility)

- 요청을 처리할 수 있는 객체가 둘 이상 존재하여 한 객체가 처리하지 못하면 다음 객체로 넘어가는 형태의 패턴이다.
- 요청을 처리할 수 있는 각 객체들이 고리(Chain)로 묶여 있어 요청이 해결될 때까지 고리를 따라 책임이 넘어간다.

필기 20.8

커맨드(Command)

- 요청을 객체의 형태로 캡슐화하여 재이용하거나 취소할 수 있도록 요청에 필요한 정보를 저장하거나 로그에 남기는 패턴이다.
- 요청에 사용되는 각종 명령어들을 추상 클래스와 구체 클래스로 분리하여 단순화한다.

인터프리터(Interpreter)

- 언어에 문법 표현을 정의하는 패턴이다.
- SQL이나 통신 프로토콜과 같은 것을 개발할 때 사용한다.

24.7

반복자(Iterator)

- 자료 구조와 같이 접근이 잦은 객체에 대해 동일한 인터페이스를 사용하도록 하는 패턴이다.
- 내부 표현 방법의 노출 없이 순차적인 접근이 가능하다.

필기 23.2, 21.5

중재자(Mediator)

- 수많은 객체들 간의 복잡한 상호작용(Interface)을 캡슐화하여 객체로 정의하는 패턴이다.
- 객체 사이의 의존성을 줄여 결합도를 감소시킬 수 있다.

메멘토(Memento)

- 특정 시점에서의 객체 내부 상태를 객체화함으로써 이후 요청에 따라 객체를 해당 시점의 상태로 돌릴 수 있는 기능을 제공하는 패턴이다.
- **Ctrl+Z**와 같은 되돌리기 기능을 개발할 때 주로 이용한다.

필기 22.10, 20.7, 필기 20.8

옵서버(Observer)

- 한 객체의 상태가 변화하면 객체에 상속되어 있는 다른 객체들에게 변화된 상태를 전달하는 패턴이다.
- 일대다의 의존성을 정의한다.
- 주로 분산된 시스템 간에 이벤트를 생성·발행(Publish)하고, 이를 수신(Subscribe)해야 할 때 이용한다.

필기 20.8

상태(State)

- 객체의 상태에 따라 동일한 동작을 다르게 처리해야 할 때 사용하는 패턴이다.
- 객체 상태를 캡슐화하고 이를 참조하는 방식으로 처리한다.

필기 24.2, 21.8

전략(Strategy)

- 동일한 계열의 알고리즘들을 개별적으로 캡슐화하여 상호 교환할 수 있게 정의하는 패턴이다.
- 클라이언트는 독립적으로 원하는 알고리즘을 선택하여 사용할 수 있으며, 클라이언트에 영향 없이 알고리즘의 변경이 가능하다.

필기 23.2

템플릿 메소드(Template Method)

- 상위 클래스에서 골격을 정의하고, 하위 클래스에서 세부 처리를 구체화하는 구조의 패턴이다.
- 유사한 서브 클래스를 묶어 공통된 내용을 상위 클래스에서 정의함으로써 코드의 양을 줄이고 유지보수를 용이하게 해준다.

필기 25.5, 20.6

방문자(Visitor)

- 각 클래스들의 데이터 구조에서 처리 기능을 분리하여 별도의 클래스로 구성하는 패턴이다.
- 분리된 처리 기능은 각 클래스를 방문(Visit)하여 수행한다.

5장 인터페이스 구현



340216

필기 25.8, 25.2, 24.7, 24.5, 24.2, 20.8, 20.6



117 요구사항 검증 방법



필기 25.8, 25.2, 24.7, 24.5, 24.2, 20.8, 20.6

요구사항 검토(Requirements Review)

- 요구사항 명세서의 오류 확인 및 표준 준수 여부 등의 결함 여부를 검토 담당자들이 수작업으로 분석하는 방법이다.
- 동료검토(Peer Review) : 요구사항 명세서 작성자가 명세서 내용을 직접 설명하고 동료들이 이를 들으면서 결함을 발견하는 형태의 검토 방법
- 워크스루(Walk Through) : 검토 회의 전에 요구사항 명세서를 미리 배포하여 사전 검토한 후에 짧은 검토 회의를 통해 결함을 발견하는 형태의 검토 방법
- 인스펙션(Inspection) : 요구사항 명세서 작성자를 제외한 다른 검토 전문가들이 요구사항 명세서를 확인하면서 결함을 발견하는 형태의 검토 방법

프로토타이핑(Prototyping)

사용자의 요구사항을 정확히 파악하기 위해 실제 개발될 소프트웨어에 대한 견본품(Prototype)을 만들어 최종 결과물을 예측한다.

테스트 설계

요구사항은 테스트할 수 있도록 작성되어야 하며, 이를 위해 테스트 케이스를 생성하여 요구사항이 현실적으로 테스트 가능한지를 검토한다.

CASE 도구 활용

일관성 분석(Consistency Analysis)을 통해 요구사항 변경사항의 추적, 분석, 관리, 표준 준수 여부를 확인한다.

340222

필기 25.2, 24.7, 24.2, 22.7, 21.8, 21.3, 20.9, 20.8



118 미들웨어



- 미들웨어(Middleware)는 운영체제와 응용 프로그램, 또는 서버와 클라이언트 사이에서 다양한 서비스를 제공하는 소프트웨어이다.
- 미들웨어는 표준화된 인터페이스를 제공함으로써 시스템 간의 데이터 교환에 일관성을 보장한다.
- 미들웨어의 종류 : DB, RPC, MOM, TP-Monitor, ORB, WAS

340223

필기 25.8, 23.7, 23.5, 23.2, 22.4, 21.3, 20.6



119 미들웨어의 종류



필기 25.8, 23.2

DB(DataBase)

- 데이터베이스 벤더에서 제공하는 클라이언트에서 원격의 데이터베이스와 연결하는 미들웨어이다.
- DB를 사용하여 시스템을 구축하는 경우 보통 2-Tier 아키텍처라고 한다.

필기 23.7, 21.3

RPC(Remote Procedure Call, 원격 프로시저 호출)

응용 프로그램의 프로시저를 사용하여 원격 프로시저를 마치 로컬 프로시저처럼 호출하는 미들웨어이다.

필기 25.8, 23.7, 23.2, 22.4

MOM(Message Oriented Middleware, 메시지 지향 미들웨어)

- 메시지 기반의 비동기형 메시지를 전달하는 미들웨어이다.
- 온라인 업무보다는 이기종 분산 데이터 시스템의 데이터 동기를 위해 많이 사용된다.

필기 25.8, 23.5, 20.6

TP-Monitor(Transaction Processing Monitor, 트랜잭션 처리 모니터)

- 온라인 트랜잭션 업무에서 트랜잭션을 처리 및 감시하는 미들웨어이다.

- 항공기나 철도 예약 업무 등 사용자 수가 증가해도 빠른 응답 속도를 유지해야 하는 업무에 주로 사용된다.

필기 25.8, 23.2

ORB(Object Request Broker, 객체 요청 브로커)

- 코바(CORBA) 표준 스펙을 구현한 객체 지향 미들웨어이다.
- 최근에는 TP-Monitor의 장점인 트랜잭션 처리와 모니터링 등을 추가로 구현한 제품도 있다.

필기 25.8, 23.7, 23.2

WAS(Web Application Server)

- 사용자의 요구에 따라 변하는 동적인 콘텐츠를 처리하기 위한 미들웨어이다.
- 클라이언트/서버 환경보다는 웹 환경을 구현하기 위한 미들웨어이다.
- HTTP 세션 처리를 위한 웹 서버 기능뿐만 아니라 미션-크리티컬한 기업 업무까지 JAVA, EJB 컴포넌트 기반으로 구현이 가능하다.

340225



120 EAI



21.4, 20.10, 필기 23.2, 22.7, 21.8, 20.9

EAI(Enterprise Application Integration)는 기업 내 각종 애플리케이션 및 플랫폼 간의 정보 전달, 연계, 통합 등 상호 연동이 가능하게 해주는 솔루션이다.

20.10, 필기 23.2, 22.7, 20.6

Point-to-Point

- 가장 기본적인 애플리케이션 통합 방식이다.
- 애플리케이션을 1:1로 연결한다.
- 변경 및 재사용이 어렵다.

20.10, 필기 23.2, 22.7, 20.6

Hub & Spoke

- 단일 접점인 허브 시스템을 통해 데이터를 전송하는 중앙 집중형 방식이다.
- 확장 및 유지 보수가 용이하다.
- 허브 장애 발생 시 시스템 전체에 영향을 미친다.

20.10, 필기 23.2, 22.7, 21.8, 20.6

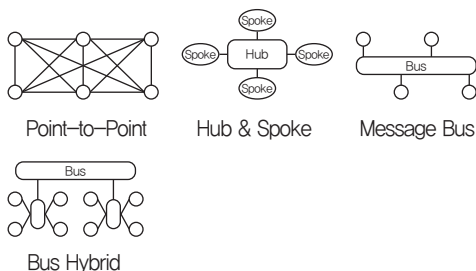
Message Bus(ESB 방식)

- 애플리케이션 사이에 미들웨어를 두어 처리하는 방식이다.
- 확장성이 뛰어나며 대용량 처리가 가능하다.

20.10, 필기 20.9

Bus Hybrid

- Hub & Spoke와 Message Bus의 혼합 방식이다.
- 그룹 내에서는 Hub & Spoke 방식을, 그룹 간에는 Message Bus 방식을 사용한다.
- 필요한 경우 한 가지 방식으로 EAI 구현이 가능하다.
- 데이터 병목 현상을 최소화할 수 있다.



340229



121 JSON



20.5, 필기 24.5, 24.2, 23.5, 22.4, 20.6

- JSON(JavaScript Object Notation)은 웹과 컴퓨터 프로그램에서 용량이 적은 데이터를 교환하기 위해 데이터 객체를 속성·값의 쌍(Attribute-Value Pairs) 형태로 표현하는 개방형 표준 포맷이다.
- 비동기 처리에 사용되는 AJAX에서 XML을 대체하여 사용되고 있다.

**122** AJAX

- AJAX(Asynchronous JavaScript and XML)는 자바 스크립트(JavaScript)를 사용하여 클라이언트와 서버 간에 XML 데이터를 주고 받는 비동기 통신 기술이다.
- 전체 페이지를 새로 고치지 않고도 웹 페이지 일부 영역만을 업데이트할 수 있다.

**123** 인터페이스 보안 기능 적용 **(B)**

- 인터페이스 보안은 인터페이스의 보안성 향상을 위해 인터페이스의 보안 취약점을 분석한 후 적절한 보안 기능을 적용하는 것이다.
- 인터페이스 보안 기능은 일반적으로 네트워크, 애플리케이션, 데이터베이스 영역에 적용한다.

필기 20.9, 20.8, 20.6 네트워크 영역	• 인터페이스 송·수신 간 스니핑(Sniffing) 등을 이용한 데이터 탈취 및 변조 위협을 방지하기 위해 네트워크 트래픽에 대한 암호화를 설정함 • 암호화는 인터페이스 아키텍처에 따라 IPsec, SSL, S-HTTP 등의 다양한 방식으로 적용함
애플리케이션 영역	소프트웨어 개발 보안 가이드를 참조하여 애플리케이션 코드 상의 보안 취약점을 보완하는 방향으로 애플리케이션 보안 기능을 적용함
데이터베이스 영역	데이터베이스, 스키마, 엔티티의 접근 권한과 프로시저(Procedure), 트리거(Trigger) 등 데이터베이스 동작 객체의 보안 취약점에 보안 기능을 적용함

**124** IPsec / SSL / S-HTTP **(A)**

- IPsec(IP Security)
 - 네트워크 계층에서 IP 패킷 단위의 데이터 변조 방지 및 은닉 기능을 제공하는 프로토콜이다.
 - 암호화 수행 시 암호화와 복호화가 모두 가능한 양방향 암호화 방식을 사용한다.
 - 구성 요소 : AH, ESP
 - 운영 모드 : 터널 모드, 전송 모드
- SSL(Secure Sockets Layer) : TCP/IP 계층과 애플리케이션 계층 사이에서 인증, 암호화, 무결성을 보장하는 프로토콜
- S-HTTP(Secure Hypertext Transfer Protocol) : 클라이언트와 서버 간에 전송되는 모든 메시지를 암호화하는 프로토콜

**125** 데이터 무결성 검사 도구 **(C)**

- 데이터 무결성 검사 도구는 인터페이스 보안 취약점을 분석하는데 사용되는 도구이다.
- 데이터 무결성 검사 도구는 시스템 파일의 변경 유무를 확인하고, 파일이 변경되었을 경우 이를 관리자에게 알려준다.
- 종류 : Tripwire, AIDE, Samhain, Claymore, Slipwire, Fcheck 등



인터페이스 구현을 검증하기 위해서는 인터페이스 단위 기능과 시나리오 등을 기반으로 하는 통합 테스트가 필요하며, 통합 테스트를 수행하기 위해 사용하는 테스트 자동화 도구는 다음과 같다.

22.5, 필기 24.7, 24.5, 24.2, 23.7, 23.2, 22.7, 21.5, 20.9

xUnit

- 같은 테스트 코드를 여러 번 작성하지 않게 도와주며, 테스트마다 예상 결과를 기억할 필요가 없게 하는 자동화된 해법을 제공하는 단위 테스트 프레임워크이다.
- Smalltalk에 처음 적용되어 SUnit이라는 이름이었으나, Java용의 JUnit, C++용의 CppUnit, .NET용의 NUnit 등 다양한 언어에 적용되면서 xUnit으로 통칭되고 있다.

필기 24.7, 24.5, 24.2, 23.2, 22.4, 21.5, 20.9, 20.6

STAF

- 서비스 호출 및 컴포넌트 재사용 등 다양한 환경을 지원하는 테스트 프레임워크이다.
- 크로스 플랫폼이나 분산 소프트웨어에서 테스트 환경을 조성할 수 있도록 지원한다.
- 분산 소프트웨어의 경우 각 분산 환경에 설치된 데몬(Daemon)이 프로그램 테스트에 대한 응답을 대신하며, 테스트가 완료되면 이를 통합하고 자동화하여 프로그램을 완성한다.

필기 24.7, 24.5, 23.7, 23.2

FitNesse

웹 기반 테스트 케이스 설계, 실행, 결과 확인 등을 지원하는 테스트 프레임워크이다.

필기 24.7, 24.5, 24.2, 23.7, 23.2, 22.4, 20.9

NTAF

FitNesse의 장점인 협업 기능과 STAF의 장점인 재사용 및 확장성을 통합한 NHN(Naver)의 테스트 자동화 프레임워크이다.

Selenium

다양한 브라우저 및 개발 언어를 지원하는 웹 애플리케이션 테스트 프레임워크이다.

필기 23.7, 21.5

watir

인터프리터 방식의 객체 지향 스크립트 언어인 Ruby를 사용하는 애플리케이션 테스트 프레임워크이다.

6장 화면 설계



340237

21.7, 필기 21.8, 21.5



127 UI / UX



21.7, 필기 21.8, 21.5

UI(User Interface, 사용자 인터페이스)

- UI는 사용자와 시스템 간의 상호작용이 원활하게 이뤄지도록 도와주는 장치나 소프트웨어를 의미한다.
- UI의 세 가지 분야
 - 정보 제공과 전달을 위한 물리적 제어에 관한 분야
 - 콘텐츠의 상세적인 표현과 전체적인 구성에 관한 분야
 - 모든 사용자가 편리하고 간편하게 사용하도록 하는 기능에 관한 분야

UX(User Experience, 사용자 경험)

- UX는 사용자가 시스템이나 서비스를 이용하면서 느끼고 생각하게 되는 총체적인 경험을 의미한다.
- UI가 사용성, 접근성, 편의성을 중시한다면 UX는 이러한 UI를 통해 사용자가 느끼는 만족이나 감정을 중시한다.

440190

22.5, 21.10, 필기 23.7, 22.7, 22.4, 21.8



128 UI의 구분



- CLI(Command Line Interface) : 명령과 출력이 텍스트 형태로 이뤄지는 인터페이스
- GUI(Graphical User Interface) : 아이콘이나 메뉴를 마우스로 선택하여 작업을 수행하는 그래픽 환경의 인터페이스
- NUI(Natural User Interface) : 사용자의 말이나 행동 등 자연스러운 움직임을 통해 기기를 조작하는 인터페이스

340239

20.10, 20.7, 필기 22.7, 20.8, 20.6



129 UI의 기본 원칙



- 직관성 : 누구나 쉽게 이해하고 사용할 수 있어야 함
- 유효성 : 사용자의 목적을 정확하고 완벽하게 달성해야 함
- 학습성 : 누구나 쉽게 배우고 익힐 수 있어야 함
- 유연성 : 사용자의 요구사항을 최대한 수용하고 실수를 최소화해야 함

340249

필기 24.7, 24.5, 21.8, 21.3, 20.8, 20.6



130 ISO/IEC 9126의 소프트웨어 품질 특성



필기 20.6

기능성(Functionality)

- 소프트웨어가 사용자의 요구사항을 정확하게 만족하는 기능을 제공하는지 여부를 나타낸다.
- 하위 특성 : 적절성/적합성, 정밀성/정확성, 상호 운용성, 보안성, 준수성

필기 20.8

신뢰성(Reliability)

- 주어진 시간동안 주어진 기능을 오류 없이 수행할 수 있는 정도를 나타낸다.
- 하위 특성 : 성숙성, 고장 허용성, 회복성

필기 21.3

사용성(Usability)

- 사용자와 컴퓨터 사이에 발생하는 어떠한 행위에 대하여 사용자가 정확하게 이해하고 사용하며, 향후 다시 사용하고 싶은 정도를 나타낸다.
- 하위 특성 : 이해성, 학습성, 운용성, 친밀성

효율성(Efficiency)

- 사용자가 요구하는 기능을 얼마나 빠르게 처리할 수 있는지 정도를 나타낸다.
- 하위 특성 : 시간 효율성, 자원 효율성

유지 보수성(Maintainability)

- 환경의 변화 또는 새로운 요구사항이 발생했을 때 소프트웨어를 개선하거나 확장할 수 있는 정도를 나타낸다.
- 하위 특성 : 분석성, 변경성, 안정성, 시험성

필기 24.7, 24.5, 21.8

이식성(Portability)

- 소프트웨어가 다른 환경에서도 얼마나 쉽게 적용할 수 있는지 정도를 나타낸다.
- 하위 특성 : 적응성, 설치성, 대체성, 공존성



340254

20.5, 필기 25.5, 24.7, 24.2, 22.7, 20.6



131 애플리케이션 테스트의 기본 원리

(A)

- 파레토 법칙(Pareto Principle) : 애플리케이션의 20%에 해당하는 코드에서 전체 결함의 80%가 발견된다는 법칙
- 살충제 패러독스(Pesticide Paradox) : 동일한 테스트 케이스로 동일한 테스트를 반복하면 더 이상 결함이 발견되지 않는 현상
- 오류-부재의 궤변(Absence of Errors Fallacy) : 소프트웨어의 결함을 모두 제거해도 사용자의 요구사항을 만족시키지 못하면 해당 소프트웨어는 품질이 높다고 말할 수 없는 것

340255

20.7



132 프로그램 실행 여부에 따른 테스트

(A)

20.7

정적 테스트

- 프로그램을 실행하지 않고 명세서나 소스 코드를 대상으로 분석하는 테스트이다.
- 소스 코드에 대한 코딩 표준, 코딩 스타일, 코드 복잡도, 남은 결함 등을 발견하기 위해 사용한다.
- 종류 : 워크스루, 인스펙션, 코드 검사 등

동적 테스트

- 프로그램을 실행하여 오류를 찾는 테스트이다.
- 소프트웨어 개발의 모든 단계에서 테스트를 수행한다.
- 종류 : 블랙박스 테스트, 화이트박스 테스트

340260

필기 24.2, 23.7, 22.3, 21.5, 20.8, 20.6



133 화이트박스 테스트의 종류

(B)

필기 24.2, 23.7, 22.3, 21.5, 20.8, 20.6

기초 경로 검사(Base Path Testing)

- 테스트 케이스 설계자가 절차적 설계의 논리적 복잡성을 측정할 수 있게 해주는 테스트 기법이다.
- 대표적인 화이트박스 테스트 기법이다.

필기 22.3

제어 구조 검사(Control Structure Testing)

- 조건 검사(Condition Testing) : 프로그램 모듈 내에 있는 논리적 조건을 테스트하는 테스트 케이스 설계 기법
- 루프 검사(Loop Testing) : 프로그램의 반복(Loop) 구조에 초점을 맞춰 실시하는 테스트 케이스 설계 기법
- 데이터 흐름 검사(Data Flow Testing) : 프로그램에서 변수의 정의와 변수 사용의 위치에 초점을 맞춰 실시하는 테스트 케이스 설계 기법

340540

25.11, 25.7, 25.4, 24.10, 24.4, 23.7, 23.4, 21.7, 20.10



134 화이트박스 테스트의 검증 기준

(A)

24.10, 21.7

문장 검증 기준(Statement Coverage)

소스 코드의 모든 구문이 한 번 이상 수행되도록 테스트 케이스를 설계한다.

25.7, 25.4, 24.10, 23.4, 21.7, 20.10

결정 검증 기준(Decision Coverage)

- 소스 코드의 모든 조건문에 대해 조건식의 결과가 True인 경우와 False인 경우가 한 번 이상 수행되도록 테스트 케이스를 설계한다.
- 분기 검증 기준(Branch Coverage)이라고도 한다.

25.11, 24.10, 23.7, 21.7

조건 검증 기준(Condition Coverage)

소스 코드의 조건문에 포함된 개별 조건식의 결과가 True인 경우와 False인 경우가 한 번 이상 수행되도록 테스트 케이스를 설계한다.

조건/결정 검증 기준(Condition/Decision Coverage)

결정 검증 기준과 조건 검증 기준을 모두 만족하는 설계로, 조건문이 True인 경우와 False인 경우에 따라 조건 검증 기준의 입력 데이터를 구분하는 테스트 케이스를 설계한다.

변경 조건/결정 검증 기준(Modified Condition/Decision Coverage)

조건/결정 검증 기준을 향상시킨 검증 기준으로, 개별 조건식이 다른 개별 조건식의 영향을 받지 않고 전체 조건식의 결과에 독립적으로 영향을 주도록 테스트 케이스를 설계한다.

다중 조건 검증 기준(Multiple Condition Coverage)

소스 코드의 조건문에 포함된 모든 개별 조건식의 모든 조합을 고려하도록 테스트 케이스를 설계한다.

340262



135 블랙박스 테스트

20.10



- 블랙박스 테스트(Black Box Test)는 소프트웨어가 수행할 특정 기능을 알기 위해서 각 기능이 완전히 작동되는 것을 입증하는 테스트로, 기능 테스트라고도 한다.
- 사용자의 요구사항 명세를 보면서 테스트한다.
- 주로 구현된 기능을 테스트한다.
- 소프트웨어 인터페이스를 통해 실시된다.

340263



23.10, 22.10, 22.5, 21.10, 21.4, 20.11, 필기 25.8, 25.5, 25.2, 24.7, 24.5, 23.5, ...

136 블랙박스 테스트의 종류



23.10, 22.5, 21.4, 20.11, 필기 25.8, 25.5, 24.7, 24.5, 23.5, 20.9, 20.8

동치 분할 검사(Equivalence Partitioning Testing)

- 프로그램의 입력 조건에 타당한 입력 자료와 타당하지 않은 입력 자료의 개수를 균등하게 하여 테스트 케이스를 정하고, 해당 입력 자료에 맞는 결과가 출력되는지 확인하는 기법이다.
- 동등 분할 기법 또는 동치 클래스 분해라고도 한다.

22.10, 22.5, 21.4, 필기 25.8, 25.5, 25.2, 24.7, 21.3, 20.9, 20.8, 20.6

경계값 분석(Boundary Value Analysis)

입력 조건의 중간값보다 경계값에서 오류가 발생할 확률이 높다는 점을 이용하여 입력 조건의 경계값을 테스트 케이스로 선정하여 검사하는 기법이다.

22.5, 21.10, 필기 20.9

원인-효과 그래프 검사(Cause-Effect Graphing Testing)

입력 데이터 간의 관계와 출력에 영향을 미치는 상황을 체계적으로 분석한 다음 효용성이 높은 테스트 케이스를 선정하여 검사하는 기법이다.

필기 25.5, 24.7, 20.8

오류 예측 검사(Error Guessing)

과거의 경험이나 확인자의 감각으로 테스트하는 기법이다.

비교 검사(Comparison Testing)

여러 버전의 프로그램에 동일한 테스트 자료를 제공하여 동일한 결과가 출력되는지 테스트하는 기법이다.

예제 A 애플리케이션에서 평가점수에 따른 성적부여 기준이 다음과 같을 때, 동치 분할 검사와 경계값 분석의 테스트 케이스를 확인시오.

평가점수	성적
90~100	A
80~89	B
70~79	C
0~69	D

〈동치 분할 검사〉

테스트 케이스	1	2	3	4
입력값	60	75	82	96
예상 결과값	D	C	B	A
실제 결과값	D	C	B	A

해설 동치 분할 검사는 입력 자료에 초점을 맞춰 테스트 케이스를 만들어 검사하므로 평가점수를 입력한 후 점수에 맞는 성적이 출력되는지 확인한다.

〈경계값 분석〉

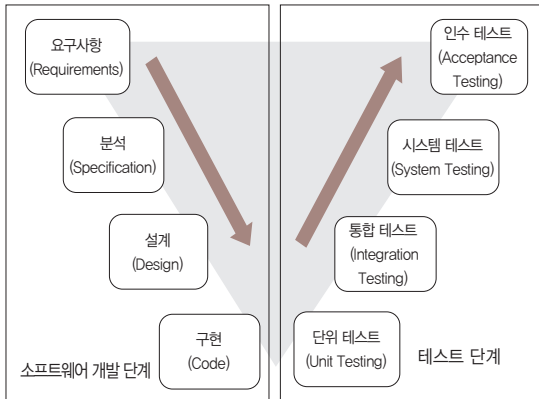
테스트 케이스	1	2	3	4	5	6	7	8	9	10
입력값	-1	0	69	70	79	80	89	90	100	101
예상 결과값	오류	D	D	C	C	B	B	A	A	오류
실제 결과값	오류	D	D	C	C	B	B	A	A	오류

해설 경계값 분석은 입력 조건의 경계값을 테스트 케이스로 선정하여 검사하므로 평가점수의 경계값에 해당하는 점수를 입력한 후 올바른 성적이 출력되는지 확인한다.



137 개발 단계에 따른 애플리케이션 테스트 (A)

- 소프트웨어의 개발 단계에 따라 단위 테스트, 통합 테스트, 시스템 테스트, 인수 테스트로 분류된다. 이렇게 분류된 것을 테스트 레벨이라고 한다.
- 애플리케이션 테스트와 소프트웨어 개발 단계를 연결하여 표현한 것을 V-모델이라고 한다.



소프트웨어 생명 주기의 V-모델



138 단위 테스트 (A)

- 단위 테스트(Unit Test)는 코딩 직후 소프트웨어 설계의 최소 단위인 모듈이나 컴포넌트에 초점을 맞춰 테스트하는 것이다.
- 인터페이스, 외부적 I/O, 자료 구조, 독립적 기초 경로, 오류 처리 경로, 경계 조건 등을 검사한다.
- 사용자의 요구사항을 기반으로 한 기능성 테스트를 최우선으로 수행한다.
- 구조 기반 테스트와 명세 기반 테스트로 나뉘지만 주로 구조 기반 테스트를 시행한다.



139 통합 테스트 (A)

- 통합 테스트(Integration Test)는 단위 테스트가 완료된 모듈들을 결합하여 하나의 시스템으로 완성시키는 과정에서의 테스트를 의미한다.
- 모듈 간 또는 통합된 컴포넌트 간의 상호 작용 오류를 검사한다.
- 비점진적 통합 방식
 - 단계적으로 통합하는 절차 없이 모든 모듈이 미리 결합되어 있는 프로그램 전체를 테스트하는 방법
 - 종류: 빅뱅 통합 테스트 방식
- 점진적 통합 방식
 - 모듈 단위로 단계적으로 통합하면서 테스트하는 방법
 - 종류: 하향식 통합 테스트, 상향식 통합 테스트, 혼합식 통합 테스트



140 인수 테스트 (A)

- 인수 테스트(Acceptance Test)는 개발한 소프트웨어가 사용자의 요구사항을 충족하는지에 중점을 두고 테스트하는 것이다.
- 인수 테스트는 개발한 소프트웨어를 사용자가 직접 테스트한다.
- 인수 테스트 종류

22.7, 필기 23.7, ... 알파 테스트

- 개발자의 장소에서 사용자가 개발자 앞에서 행하는 테스트 기법
- 테스트는 통제된 환경에서 행해지며, 오류와 사용상의 문제점을 사용자와 개발자가 함께 확인하면서 기록함

22.7, 필기 22.3, ... 베타 테스트

- 선정된 최종 사용자가 여러 명의 사용자 앞에서 행하는 테스트 기법
- 실업무를 가지고 사용자가 직접 테스트



141 하향식 통합 테스트

(A)

- 하향식 통합 테스트(Top Down Integration Test)는 프로그램의 상위 모듈에서 하위 모듈 방향으로 통합하면서 테스트하는 기법이다.
- 하향식 통합 테스트 절차
 - ① 주요 제어 모듈은 작성된 프로그램을 사용하고, 주요 제어 모듈의 종속 모듈들은 스텝(Stub)으로 대체한다.
 - ② 깊이 우선 또는 넓이 우선 등의 통합 방식에 따라 하위 모듈인 스텝들이 한 번에 하나씩 실제 모듈로 교체된다.
 - ③ 모듈이 통합될 때마다 테스트를 실시한다.
 - ④ 새로운 오류가 발생하지 않음을 보증하기 위해 회귀 테스트를 실시한다.

※ 스텝(Stub) : 제어 모듈이 호출하는 타 모듈의 기능을 단순히 수행하는 도구로, 일시적으로 필요한 조건만을 가지고 있는 시험용 모듈임



142 상향식 통합 테스트

(A)

- 상향식 통합 테스트(Bottom Up Integration Test)는 프로그램의 하위 모듈에서 상위 모듈 방향으로 통합하면서 테스트하는 기법이다.
- 상향식 통합 테스트 절차
 - ① 하위 모듈들을 클러스터(Cluster)로 결합한다.
 - ② 상위 모듈에서 데이터의 입·출력을 확인하기 위해 더미 모듈인 드라이버(Driver)를 작성한다.
 - ③ 통합된 클러스터 단위로 테스트한다.
 - ④ 테스트가 완료되면 클러스터는 프로그램 구조의 상위로 이동하여 결합하고 드라이버는 실제 모듈로 대체된다.
- ※ 테스트 드라이버(Test Driver) : 테스트 대상의 하위 모듈을 호출하고, 파라미터를 전달하고, 모듈 테스트 수행 후의 결과를 도출하는 도구임



143 회귀 테스트

(A)

- 회귀 테스트(Regression Test)는 통합 테스트로 인해 변경된 모듈이나 컴포넌트에 새로운 오류가 있는지 확인하는 테스트이다.
- 이미 테스트된 프로그램의 테스트를 반복하는 것이다.
- 회귀 테스트는 수정한 모듈이나 컴포넌트가 다른 부분에 영향을 미치는지, 오류가 생기지 않았는지 테스트하여 새로운 오류가 발생하지 않음을 보증하기 위해 반복 테스트한다.



144 테스트 오라클

(C)

- 테스트 오라클(Test Oracle)은 테스트 결과가 올바른지 판단하기 위해 사전에 정의된 참값을 대입하여 비교하는 기법 및 활동을 말한다.
- 결과를 판단하기 위해 테스트 케이스에 대한 예상 결과를 계산하거나 확인한다.
- 테스트 오라클의 특징 : 제한된 검증, 수학적 기법, 자동화 기능



145 테스트 오라클의 종류

(A)

- 참(True) 오라클 : 모든 테스트 케이스의 입력 값에 대해 기대하는 결과를 제공하는 오라클로, 발생된 모든 오류를 검출할 수 있음
- 샘플링(Sampling) 오라클 : 특정한 몇몇 테스트 케이스의 입력 값들에 대해서만 기대하는 결과를 제공하는 오라클로 전수 테스트가 불가능한 경우 사용하는 오라클
- 추정(Heuristic) 오라클 : 특정 테스트 케이스의 입력 값에 대해 기대하는 결과를 제공하고, 나머지 입력 값들에 대해서는 추정으로 처리하는 오라클
- 일관성 검사(Consistent) 오라클 : 애플리케이션에 변경이 있을 때, 테스트 케이스의 수행 전과 후의 결과 값이 동일한지를 확인하는 오라클



146 애플리케이션 성능 측정 지표 (A)

- 처리량(Throughput) : 일정 시간 내에 애플리케이션이 처리하는 일의 양
- 응답 시간(Response Time) : 애플리케이션에 요청을 전달한 시간부터 응답이 도착할 때까지 걸린 시간
- 경과 시간(Turn Around Time) : 애플리케이션에 작업을 의뢰한 시간부터 처리가 완료될 때까지 걸린 시간
- 자원 사용률(Resource Usage) : 애플리케이션이 의뢰한 작업을 처리하는 동안의 CPU 사용량, 메모리 사용량, 네트워크 사용량 등 자원 사용률



147 빅오 표기법으로 표현한 최악의 시간 복잡도 (B)

필기 23.7, 22.7, ... O(1)	입력값(n)에 관계 없이 일정하게 문제 해결에 하나의 단계만을 거침 예) 스택의 삽입(Push), 삭제(Pop)
O(log₂n)	문제 해결에 필요한 단계가 입력값(n) 또는 조건에 의해 감소함 예) 이진 트리(Binary Tree), 이진 검색(Binary Search)
O(n)	문제 해결에 필요한 단계가 입력값(n)과 1:1의 관계를 가짐 예) for문
필기 25.5, 24.7, ... O(nlog₂n)	문제 해결에 필요한 단계가 n(log ₂ n)번만큼 수행됨 예) 힙 정렬(Heap Sort), 2-Way 합병 정렬(Merge Sort)
O(n²)	문제 해결에 필요한 단계가 입력값(n)의 제곱만큼 수행됨 예) 삽입 정렬(Insertion Sort), 쉘 정렬(Shell Sort), 선택 정렬(Selection Sort), 버블 정렬(Bubble Sort), 퀵 정렬(Quick Sort)
O(2ⁿ)	문제 해결에 필요한 단계가 2의 입력값(n) 제곱만큼 수행됨 예) 피보나치 수열(Fibonacci Sequence)



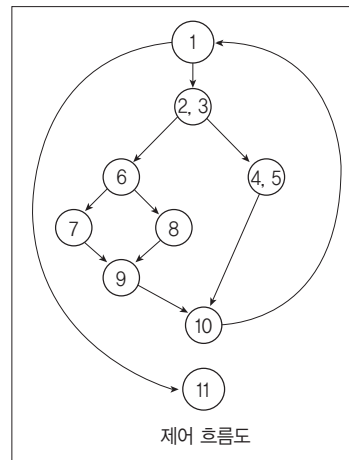
148 순환 복잡도 (C)

- 순환 복잡도(Cyclomatic Complexity)는 한 프로그램의 논리적인 복잡도를 측정하기 위한 소프트웨어의 척도이다.
- 맥케이크 순환도(McCabe's Cyclomatic)라고도 한다.
- 제어 흐름도 G에서 순환 복잡도 V(G)는 다음과 같은 방법으로 계산할 수 있다.

방법 1 순환 복잡도는 제어 흐름도의 영역 수와 일치하므로 영역 수를 계산한다.

방법 2 $V(G) = E - N + 2$: E는 화살표 수, N은 노드의 수

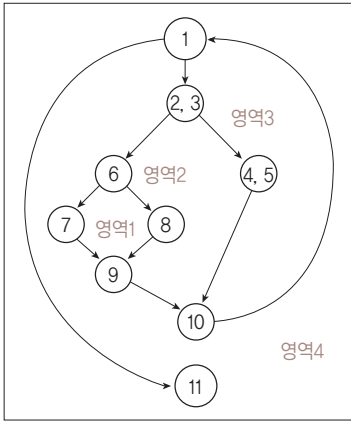
예제 제어 흐름도가 다음과 같을 때 순환 복잡도(Cyclomatic Complexity)를 계산하시오.



해설

순환 복잡도는 다음 두 가지 방법으로 구할 수 있습니다.

방법 1 제어 흐름도에서 화살표로 구분되는 각 영역의 개수를 구하면 4입니다.



방법 2 순환 복잡도 = 화살표의 수 - 노드의 수 + 2
이므로 $11 - 9 + 2 = 4$ 입니다.



149 소스 코드 최적화



- 소스 코드 최적화는 나쁜 코드(Bad Code)를 배제하고, 클린 코드(Clean Code)로 작성하는 것이다.
- 나쁜 코드로 작성된 애플리케이션의 코드를 클린 코드로 수정하면 애플리케이션의 성능이 개선된다.
- 클린 코드(Clean Code) : 누구나 쉽게 이해하고 수정 및 추가할 수 있는 단순, 명료한 코드, 즉 잘 작성된 코드
- 나쁜 코드(Bad Code)
 - 프로그램의 로직(Logic)이 복잡하고 이해하기 어려운 코드
 - 스파게티 코드(Spaghetti Code) : 코드의 로직이 서로 복잡하게 얽혀 있는 코드
 - 외계인 코드(Alien Code) : 아주 오래되거나 참고문서 또는 개발자가 없어 유지보수 작업이 어려운 코드

340290



150 클린 코드 작성 원칙



필기 24.2, 22.7, 22.3, 20.9, 20.8

필기 24.2, 22.7, 22.3

가독성

- 누구든지 코드를 쉽게 읽을 수 있도록 작성한다.
- 코드 작성 시 이해하기 쉬운 용어를 사용하거나 들여쓰기 기능 등을 사용한다.

필기 24.2, 22.7, 20.9, 20.8

단순성

- 코드를 간단하게 작성한다.
- 한 번에 한 가지를 처리하도록 코드를 작성하고 클래스/메소드/함수 등을 최소 단위로 분리한다.

필기 24.2, 22.7, 22.3, 20.8

의존성 배제

- 코드가 다른 모듈에 미치는 영향을 최소화한다.
- 코드 변경 시 다른 부분에 영향이 없도록 작성한다.

필기 24.2, 22.7, 22.3, 20.8

중복성 최소화

- 코드의 중복을 최소화한다.
- 중복된 코드는 삭제하고 공통된 코드를 사용한다.

필기 22.3

추상화

상위 클래스/메소드/함수에서는 간략하게 애플리케이션의 특성을 나타내고, 상세 내용은 하위 클래스/메소드/함수에서 구현한다.



151 소스 코드 품질 분석 도구



소스 코드 품질 분석 도구는 소스 코드의 코딩 스타일, 코드에 설정된 코딩 표준, 코드의 복잡도, 코드에 존재하는 메모리 누수 현상, 스레드 결함 등을 발견하기 위해 사용하는 분석 도구이다.

22.5, 필기 25.5, 23.7, 21.8, 20.9, 20.6

정적 분석(Static Analysis) 도구

- 작성한 소스 코드를 실행하지 않고 코딩 표준이나 코딩 스타일, 결함 등을 확인하는 코드 분석 도구이다.
- 종류 : pmd, cppcheck, SonarQube, checkstyle, ccm, cobertura 등

22.5

동적 분석(Dynamic Analysis) 도구

- 작성한 소스 코드를 실행하여 코드에 존재하는 메모리 누수, 스레드 결함 등을 분석하는 도구이다.
- 종류 : Avalanche, Valgrind 등

8장 SQL 응용



340292

필기 25.5, 25.2, 24.2, 23.7, 23.2, 21.8, 21.5, 21.3, 20.6



152 DDL



- DDL(Data Define Language, 데이터 정의어)은 DB 구조, 데이터 형식, 접근 방식 등 DB를 구축하거나 수정할 목적으로 사용하는 언어이다.
- DDL의 3가지 유형

명령어	기능
필기 23.7, 23.2, 21.8, 21.5, ... CREATE	SCHEMA, DOMAIN, TABLE, VIEW, INDEX를 정의함
필기 23.7, 23.2, 21.8, 20.6 ALTER	TABLE에 대한 정의를 변경하는 데 사용함
필기 23.7, 23.2, 21.8, 20.6 DROP	SCHEMA, DOMAIN, TABLE, VIEW, INDEX를 삭제함

340296

20.7



153 CREATE INDEX



CREATE INDEX는 인덱스를 정의하는 명령문이다.

표기 형식

```
CREATE [UNIQUE] INDEX 인덱스명
ON 테이블명(속성명 [ASC | DESC] [,속성명 [ASC | DESC]])
[CLUSTER];
```

- UNIQUE
 - 사용된 경우 : 중복 값이 없는 속성으로 인덱스를 생성함
 - 생략된 경우 : 중복 값을 허용하는 속성으로 인덱스를 생성함
- 정렬 여부 지정
 - ASC : 오름차순 정렬
 - DESC : 내림차순 정렬
 - 생략된 경우 : 오름차순으로 정렬됨

- CLUSTER : 사용하면 인덱스가 클러스터드 인덱스로 설정됨

예제 <고객> 테이블에서 UNIQUE한 특성을 갖는 '고객번호' 속성에 대해 내림차순으로 정렬하여 '고객번호_idx'라는 이름으로 인덱스를 정의하시오.

```
CREATE UNIQUE INDEX 고객번호_idx
ON 고객(고객번호 DESC);
```

340297

20.10, 필기 21.3, 20.9



154 ALTER TABLE



ALTER TABLE은 테이블에 대한 정의를 변경하는 명령문이다.

표기 형식

```
ALTER TABLE 테이블명 ADD 속성명 데이터_타입 [DEFAULT '기본값'];
ALTER TABLE 테이블명 ALTER 속성명 [SET DEFAULT '기본값'];
ALTER TABLE 테이블명 DROP COLUMN 속성명 [CASCADE];
```

- ADD : 새로운 속성(열)을 추가할 때 사용함
- ALTER : 특정 속성의 Default 값을 변경할 때 사용함
- DROP COLUMN : 특정 속성을 삭제할 때 사용함

예제 1 <학생> 테이블에 최대 3문자로 구성되는 '학년' 속성 추가하시오.

```
ALTER TABLE 학생 ADD 학년 VARCHAR(3);
```

예제 2 <학생> 테이블의 '학번' 필드의 데이터 타입과 크기를 VARCHAR(10)으로 하고 NULL 값이 입력되지 않도록 변경하시오.

```
ALTER TABLE 학생 ALTER 학번 VARCHAR(10) NOT NULL;
```

**155 DROP**

DROP은 스키마, 도메인, 기본 테이블, 뷰 테이블, 인덱스, 제약 조건 등을 제거하는 명령문이다.

표기 형식

DROP SCHEMA 스키마명 [CASCADE | RESTRICT];
 DROP DOMAIN 도메인명 [CASCADE | RESTRICT];
 DROP TABLE 테이블명 [CASCADE | RESTRICT];
 DROP VIEW 뷰명 [CASCADE | RESTRICT];
 DROP INDEX 인덱스명 [CASCADE | RESTRICT];
 DROP CONSTRAINT 제약조건명;

- CASCADE : 제거할 요소를 참조하는 다른 모든 개체를 함께 제거함
- RESTRICT : 다른 개체가 제거할 요소를 참조중일 때는 제거를 취소함

예제 <학생> 테이블을 제거하되, <학생> 테이블을 참조하는 모든 데이터를 함께 제거하시오.

DROP TABLE 학생 CASCADE;

**156 DCL**

- DCL(Data Control Language, 데이터 제어어)은 데이터의 보안, 무결성, 회복, 병행 제어 등을 정의하는 데 사용하는 언어이다.
- DCL은 데이터베이스 관리자(DBA)가 데이터를 관리할 목적으로 사용한다.

• DCL의 종류

명령어	기능
필기 20.8 COMMIT	명령에 의해 수행된 결과를 실제 물리적 디스크로 저장하고, 데이터베이스 조작 작업이 정상적으로 완료되었음을 관리자에게 알려줌
필기 21.5, 20.8 ROLLBACK	데이터베이스 조작 작업이 비정상적으로 종료되었을 때 원래의 상태로 복구함
필기 20.8 GRANT	데이터베이스 사용자에게 사용 권한을 부여함
필기 24.7 REVOKE	데이터베이스 사용자의 사용 권한을 취소함

**157 GRANT / REVOKE**

- 데이터베이스 관리자가 데이터베이스 사용자에게 권한을 부여하거나 취소하기 위한 명령어이다.
- GRANT : 권한 부여를 위한 명령어
- REVOKE : 권한 취소를 위한 명령어
- 테이블 및 속성에 대한 권한 부여 및 취소

GRANT 권한_리스트 ON 개체 TO 사용자 [WITH GRANT OPTION];
 REVOKE [GRANT OPTION FOR] 권한_리스트 ON 개체 FROM 사용자 [CASCADE];

- 권한 종류 : ALL, SELECT, INSERT, DELETE, UPDATE 등
- WITH GRANT OPTION : 부여받은 권한을 다른 사용자에게 다시 부여할 수 있는 권한을 부여함
- GRANT OPTION FOR : 다른 사용자에게 권한을 부여할 수 있는 권한을 취소함
- CASCADE : 권한 취소 시 권한을 부여받았던 사용자가 다른 사용자에게 부여한 권한도 연쇄적으로 취소함

예제1 사용자 ID가 "NABI"인 사람에게 <고객> 테이블에 대한 모든 권한과 다른 사람에게 권한을 부여할 수 있는 권한까지 부여하는 SQL문을 작성하시오.

GRANT ALL ON 고객 TO NABI WITH GRANT OPTION;

예제 2 사용자 ID가 “STAR”인 사람에게 부여한 <고객> 테이블에 대한 권한 중 UPDATE 권한을 다른 사람에게 부여할 수 있는 권한만 취소하는 SQL문을 작성하시오.

```
REVOKE GRANT OPTION FOR UPDATE ON 고객 FROM STAR;
```

340301



158 ROLLBACK

20.7, 필기 21.5



- ROLLBACK은 변경되었으나 아직 COMMIT되지 않은 모든 내용들을 취소하고 데이터베이스를 이전 상태로 되돌리는 명령어이다.
- 트랜잭션 전체가 성공적으로 끝나지 못하면 일부 변경된 내용만 데이터베이스에 반영되는 비일관성(Inconsistency) 상태가 될 수 있기 때문에 일부분만 완료된 트랜잭션은 롤백(Rollback)되어야 한다.

340303



159 DML

필기 24.7, 23.5, 20.8, 20.6



- DML(Data Manipulation Language, 데이터 조작어)은 데이터베이스 사용자가 저장된 데이터를 실질적으로 관리하는데 사용되는 언어이다.
- DML은 데이터베이스 사용자와 데이터베이스 관리 시스템 간의 인터페이스를 제공한다.
- DML의 유형

명령어	기능
필기 20.8, 20.6 SELECT	테이블에서 튜플을 검색함
필기 20.8, 20.6 INSERT	테이블에 새로운 튜플을 삽입함
필기 20.8, 20.6 DELETE	테이블에서 튜플을 삭제함
필기 23.5, 20.8, 20.6 UPDATE	테이블에서 튜플의 내용을 갱신함

※ 다음 테이블을 참조하여 핵심 160~162의 예제 결과를 확인하시오.

<사원>

이름	부서	생일	주소	기본급
홍길동	기획	04/05/61	망원동	120
임꺽정	인터넷	01/09/69	성산동	80
황진이	편집	07/21/75	연희동	100
김선달	편집	10/22/73	망원동	90
성춘향	기획	02/20/64	망원동	100
장길산	편집	03/11/67	상암동	120
일지매	기획	04/29/78	합정동	110
강호동	인터넷	12/11/80		90

340543



160 삽입문(INSERT INTO~)

24.7, 23.7, 필기 23.7, 23.5



삽입문은 기본 테이블에 새로운 튜플을 삽입할 때 사용한다.

일반 형식

```
INSERT INTO 테이블명([속성명1, 속성명2,...])
VALUES (데이터1, 데이터2,... );
```

- 대응하는 속성과 데이터는 개수와 데이터 유형이 일치해야 한다.
- 기본 테이블의 모든 속성을 사용할 때는 속성명을 생략할 수 있다.
- SELECT문을 사용하여 다른 테이블의 검색 결과를 삽입할 수 있다.

예제 1 <사원> 테이블에 (이름 – 홍승현, 부서 – 인터넷)을 삽입하시오.

```
INSERT INTO 사원(이름, 부서) VALUES ('홍승현', '인터넷');
```

예제 2 <사원> 테이블에 (장보고, 기획, 05/03/73, 홍제동, 90)을 삽입하시오.

```
INSERT INTO 사원 VALUES ('장보고', '기획',
#05/03/73#, '홍제동', 90);
```

예제 3 <사원> 테이블에 있는 편집부의 모든 튜플을 편집 부원(이름, 생일, 주소, 기본급) 테이블에 삽입하시오.

```
INSERT INTO 편집부원(이름, 생일, 주소, 기본급)
SELECT 이름, 생일, 주소, 기본급
FROM 사원
WHERE 부서 = '편집';
```

340305



161 삭제문(DELETE FROM~)

(A)

삭제문은 기본 테이블에 있는 튜플들 중에서 특정 튜플(행)을 삭제할 때 사용한다.

일반 형식

```
DELETE FROM 테이블명 [WHERE 조건];
```

- 모든 레코드를 삭제할 때는 WHERE절을 생략한다.
- 모든 레코드를 삭제하더라도 테이블 구조는 남아 있기 때문에 디스크에서 테이블을 완전히 제거하는 DROP과는 다르다.

예제 1 <사원> 테이블에서 “임꺽정”에 대한 튜플을 삭제하시오.

```
DELETE FROM 사원 WHERE 이름 = '임꺽정';
```

예제 2 <사원> 테이블에서 “인터넷” 부서에 대한 모든 튜플을 삭제하시오.

```
DELETE FROM 사원 WHERE 부서 = '인터넷';
```

예제 3 <사원> 테이블의 모든 레코드를 삭제하시오.

```
DELETE FROM 사원;
```

340306



162 갱신문(UPDATE~ SET~)

(A)

갱신문은 기본 테이블에 있는 튜플들 중에서 특정 튜플의 내용을 변경할 때 사용한다.

일반 형식

```
UPDATE 테이블명
SET 속성명 = 데이터[, 속성명=데이터, ...]
[WHERE 조건];
```

예제 1 <사원> 테이블에서 “홍길동”의 “주소”를 “수색동”으로 수정하시오.

```
UPDATE 사원 SET 주소 = '수색동' WHERE 이름 = '홍길동';
```

예제 2 <사원> 테이블에서 “황진이”의 “부서”를 “기획부”로 변경하고 “기본급”을 5만원 인상시키시오.

```
UPDATE 사원
SET 부서 = '기획', 기본급 = 기본급 + 5
WHERE 이름 = '황진이';
```

340307



163 SELECT

(A)

일반 형식

```
SELECT [PREDICATE] [테이블명.]속성명 [AS 별칭][, [테이블명.]속성명, ...]
FROM 테이블명[, 테이블명, ...]
[WHERE 조건]
[GROUP BY 속성명, 속성명, ...]
[HAVING 조건]
[ORDER BY 속성명 [ASC | DESC]];
```

• SELECT절

– PREDICATE : 검색할 튜플 수를 제한하는 명령어를 기술함

▶ DISTINCT : 중복된 튜플이 있으면 그 중 첫 번째 한 개만 표시함

– 속성명 : 검색하여 불러올 속성(열) 또는 속성을 이용한 수식을 지정함

– AS : 속성이나 연산의 이름을 다른 이름으로 표시하기 위해 사용함

• FROM절 : 검색할 데이터가 들어있는 테이블 이름을 기술함

• WHERE절 : 검색할 조건을 기술함

- GROUP BY절 : 특정 속성을 기준으로 그룹화하여 검색할 때 사용한다. 일반적으로 GROUP BY절은 그룹 함수와 함께 사용됨
- HAVING절 : GROUP BY와 함께 사용되며, 그룹에 대한 조건을 지정함
- ORDER BY절 : 데이터를 정렬하여 검색할 때 사용함
 - 속성명 : 정렬의 기준이 되는 속성명을 기술함
 - [ASC | DESC] : 정렬 방식으로, 'ASC'는 오름차순, 'DESC'는 내림차순임. 생략하면 오름차순으로 지정됨

※ 다음 테이블을 참조하여 핵심 164~167의 예제 결과를 확인하시오.

〈사원〉

이름	부서	생일	주소	기본급
홍길동	기획	04/05/61	망원동	120
임꺽정	인터넷	01/09/69	서교동	80
황진이	편집	07/21/75	합정동	100
김선달	편집	10/22/73	망원동	90
성춘향	기획	02/20/64	대흥동	100
장길산	편집	03/11/67	상암동	120
일지매	기획	04/29/78	연남동	110
강건달	인터넷	12/11/80		90

〈여가활동〉

이름	취미	경력
김선달	당구	10
성춘향	나이트댄스	5
일지매	태권	15
임꺽정	씨름	8

340308

24.7, 20.5

164
기본 검색

SELECT 절에 원하는 속성을 지정하여 검색한다.

예제 1 〈사원〉 테이블의 모든 튜플을 검색하시오.

```
SELECT * FROM 사원;
```

〈결과〉

이름	부서	생일	주소	기본급
홍길동	기획	04/05/61	망원동	120
임꺽정	인터넷	01/09/69	서교동	80
황진이	편집	07/21/75	합정동	100
김선달	편집	10/22/73	망원동	90
성춘향	기획	02/20/64	대흥동	100
장길산	편집	03/11/67	상암동	120
일지매	기획	04/29/78	연남동	110
강건달	인터넷	12/11/80		90

예제 2 〈사원〉 테이블에서 '주소'만 검색하되 같은 '주소'는 한 번만 출력하시오.

```
SELECT DISTINCT 주소 FROM 사원;
```

〈결과〉

주소
망원동
서교동
합정동
대흥동
상암동
연남동

440245

25.11, 25.4, 22.7, 21.7, 20.7, 필기 25.5, 25.2, 24.7, 23.7, 23.5, 22.3, 21.8

165
조건 지정 검색

- WHERE 절에 다음 연산자들을 이용한 조건을 지정하여 조건에 만족하는 튜플만 검색한다.
- 비교 연산자

연산자	=	<>	>	<	>=	<=
의미	같다	같지 않다	크다	작다	크거나 같다	작거나 같다

- 논리 연산자 : NOT, AND, OR
- LIKE 연산자 : 대표 문자를 이용해 지정된 속성의 값이 문자 패턴과 일치하는 튜플을 검색하기 위해 사용됨

대표 문자	%	_	#
의미	모든 문자를 대표함	문자 하나를 대표함	숫자 하나를 대표함

- IN 연산자 : 필드의 값이 IN 연산자의 수로 지정된 값과 같은 레코드만 검색하며, OR 연산을 수행한 결과와 같음

예제 1 〈사원〉 테이블에서 '기획'부의 모든 튜플을 검색하십시오.

```
SELECT * FROM 사원 WHERE 부서 = '기획';
```

〈결과〉

이름	부서	생일	주소	기본급
홍길동	기획	04/05/61	망원동	120
성춘향	기획	02/20/64	대흥동	100
일지매	기획	04/29/78	연남동	110

예제 2 〈사원〉 테이블에서 “기획” 부서에 근무하면서 “대흥동”에 사는 사람의 튜플을 검색하십시오.

```
SELECT *
FROM 사원
WHERE 부서 = '기획' AND 주소 = '대흥동';
```

〈결과〉

이름	부서	생일	주소	기본급
성춘향	기획	02/20/64	대흥동	100

예제 3 〈사원〉 테이블에서 성이 '김'인 사람의 튜플을 검색하십시오.

```
SELECT *
FROM 사원
WHERE 이름 LIKE "김%";
```

〈결과〉

이름	부서	생일	주소	기본급
김선달	편집	10/22/73	망원동	90

예제 4 〈사원〉 테이블에서 '생일'이 '01/01/69'에서 '12/31/73' 사이인 튜플을 검색하십시오.

```
SELECT *
FROM 사원
WHERE 생일 BETWEEN #01/01/69# AND #12/31/73#;
```

〈결과〉

이름	부서	생일	주소	기본급
임꺽정	인터넷	01/09/69	서교동	80
김선달	편집	10/22/73	망원동	90

예제 5 〈사원〉 테이블에서 '주소'가 NULL인 튜플을 검색하십시오.

```
SELECT *
FROM 사원
WHERE 주소 IS NULL;
```

〈결과〉

이름	부서	생일	주소	기본급
강건달	인터넷	12/11/80		90

340310



22.5, 21.7, 필기 25.2, 23.5, 22.4

166 정렬 검색



ORDER BY 절에 특정 속성을 지정하여 지정된 속성으로 자료를 정렬하여 검색한다.

예제 〈사원〉 테이블에서 '부서'를 기준으로 오름차순 정렬하고, 같은 '부서'에 대해서는 '이름'을 기준으로 내림차순 정렬시켜서 검색하십시오.

```
SELECT *
FROM 사원
ORDER BY 부서 ASC, 이름 DESC;
```

〈결과〉

이름	부서	생일	주소	기본급
홍길동	기획	04/05/61	망원동	120
일지매	기획	04/29/78	연남동	110
성춘향	기획	02/20/64	대흥동	100
임꺽정	인터넷	01/09/69	서교동	80
강건달	인터넷	12/11/80		90
황진이	편집	07/21/75	합정동	100
장길산	편집	03/11/67	상암동	120
김선달	편집	10/22/73	망원동	90

440247



24.10, 24.4, 22.7, 필기 25.8, 24.7, 24.2, 23.5, 22.4, 21.5, 20.9, 20.6

167 하위 질의



하위 질의는 조건절에 주어진 질의를 먼저 수행하여 그 검색 결과를 조건절의 피연산자로 사용한다.

예제 1 '취미'가 "나이트댄스"인 사원의 '이름'과 '주소'를 검색하시오.

```
SELECT 이름, 주소
FROM 사원
WHERE 이름 = (SELECT 이름 FROM 여가활동 WHERE 취미 = '나이트댄스');
```

〈결과〉

이름	주소
성춘향	대흥동

예제 2 취미활동을 하지 않는 직원들을 검색하시오.

```
SELECT *
FROM 사원
WHERE 이름 NOT IN (SELECT 이름 FROM 여가활동);
```

〈결과〉

이름	부서	생일	주소	기본급
홍길동	기획	04/05/61	망원동	120
황진이	편집	07/21/75	합정동	100
장길산	편집	03/11/67	상암동	120
강건달	인터넷	12/11/80		90

예제 3 "망원동"에 거주하는 직원들의 '기본급'보다 적은 '기본급'을 받는 직원의 정보를 검색하시오.

```
SELECT 이름, 기본급, 주소
FROM 사원
WHERE 기본급 < ALL (SELECT 기본급 FROM 사원 WHERE 주소 = "망원동");
```

〈결과〉

이름	기본급	주소
임꺽정	80	서교동

해설

ALL ()는 하위 질의로 검색된 범위를 기본 질의의 조건으로 사용한다. 즉 〈사원〉 테이블에서 주소가 "망원동"인 사원의 기본급을 모두 추출한 후 추출된 기본급들을 기본 질의의 조건으로 사용한다.

① SELECT 기본급 FROM 사원 WHERE 주소 = "망원동"; : 〈사원〉 테이블에서 '주소'가 "망원동"인 직원들의 '기본급'을 추출한다. 결과는 120, 90이다.

② SELECT 이름, 기본급, 주소 FROM 사원 WHERE 기본급 < ALL (①) : '기본급'이 ①에서 추출된 '기본급'의 모든(ALL) 범위인 120, 90보다 작은(<), 즉 범위 안에서 가장 작은 90보다 작은 기본급을 갖는 자료를 대상으로 '이름', '기본급', '주소'를 출력한다. 결과는 "임꺽정", "80", "서교동"이다.

340312



25.11, 23.4, 22.10, 21.4, 필기 24.5, 20.8

168 그룹 함수



그룹 함수는 GROUP BY절에 지정된 그룹별로 속성의 값을 집계할 때 사용된다.

명령어	기능
COUNT(속성명)	그룹별 튜플 수를 구하는 함수
SUM(속성명)	그룹별 합계를 구하는 함수
AVG(속성명)	그룹별 평균을 구하는 함수
MAX(속성명)	그룹별 최대값을 구하는 함수
MIN(속성명)	그룹별 최소값을 구하는 함수
STDDEV(속성명)	그룹별 표준편차를 구하는 함수
VARIANCE(속성명)	그룹별 분산을 구하는 함수



169 그룹 지정 검색



GROUP BY절에 지정한 속성을 기준으로 자료를 그룹화하여 검색한다.

〈상여금〉

부서	이름	상여내역	상여금
기획	홍길동	연장근무	100
기획	일지매	연장근무	100
기획	최준호	야간근무	120
기획	장길산	특별근무	90
인터넷	강건달	특별근무	90
인터넷	서국현	특별근무	90
인터넷	박인식	연장근무	30
편집	김선달	특별근무	80
편집	황종근	연장근무	40
편집	성춘향	야간근무	80
편집	임격정	야간근무	80
편집	황진이	야간근무	50

예제 1 〈상여금〉 테이블에서 '부서'별 '상여금'의 평균을 구하시오.

```
SELECT 부서, AVG(상여금) AS 평균
FROM 상여금
GROUP BY 부서;
```

〈결과〉

부서	평균
기획	102.5
인터넷	70
편집	66

예제 2 〈상여금〉 테이블에서 '상여금'이 100 이상인 사람이 2명 이상인 '부서'의 튜플 수를 구하시오.

```
SELECT 부서, COUNT(*) AS 인원수
FROM 상여금
WHERE 상여금 >= 100
GROUP BY 부서
HAVING COUNT(*) >= 2;
```

〈결과〉

부서	사원수
기획	3



170 집합 연산자를 이용한 통합 질의



집합 연산자를 사용하여 2개 이상의 테이블의 데이터를 하나로 통합한다.

표기 형식

```
SELECT 속성명1, 속성명2, ...
FROM 테이블명
UNION | UNION ALL | INTERSECT | EXCEPT
SELECT 속성명1, 속성명2, ...
FROM 테이블명
[ORDER BY 속성명 [ASC | DESC]];
```

- 두 개의 SELECT문에 기술한 속성들은 개수와 데이터 유형이 서로 동일해야 한다.
- 집합 연산자의 종류(통합 질의의 종류)

집합 연산자	설명	집합 종류
UNION	• 두 SELECT문의 조회 결과를 통합하여 모두 출력함 • 중복된 행은 한 번만 출력함	합집합
UNION ALL	• 두 SELECT문의 조회 결과를 통합하여 모두출력함 • 중복된 행도 그대로 출력함	합집합
INTERSECT	두 SELECT문의 조회 결과 중 공통된 행만 출력함	교집합
EXCEPT	첫 번째 SELECT문의 조회 결과에서 두 번째 SELECT문의 조회 결과를 제외한 행을 출력함	차집합



171 JOIN



- JOIN은 2개의 릴레이션에서 **연관된 튜플들을 결합하여, 하나의 새로운 릴레이션을 반환한다.**
- JOIN은 일반적으로 FROM절에 기술하지만, 릴레이션이 사용되는 곳 어디에나 사용할 수 있다.
- JOIN은 크게 INNER JOIN과 OUTER JOIN으로 구분된다.
 - INNER JOIN : THETA JOIN, EQUI JOIN, NATURAL JOIN, NON-EQUI JOIN
 - OUTER JOIN : LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN
- 조건이 없는 INNER JOIN을 수행하면 CROSS JOIN과 동일한 결과를 얻을 수 있다.
- CROSS JOIN(교차 조인)
 - 교차 조인은 조인하는 두 테이블에 있는 튜플들의 순서쌍을 결과로 반환한다.
 - 교차 조인의 결과로 반환되는 테이블의 행의 수는 두 테이블의 행 수를 곱한 것과 같다.

- WHERE절을 이용한 EQUI JOIN의 표기 형식

```
SELECT [테이블명1.]속성명, [테이블명2.]속성명, ...
FROM 테이블명1, 테이블명2, ...
WHERE 테이블명1.속성명 = 테이블명2.속성명;
```

- NATURAL JOIN절을 이용한 EQUI JOIN의 표기 형식

```
SELECT [테이블명1.]속성명, [테이블명2.]속성명, ...
FROM 테이블명1 NATURAL JOIN 테이블명2;
```

- JOIN ~ USING절을 이용한 EQUI JOIN의 표기 형식

```
SELECT [테이블명1.]속성명, [테이블명2.]속성명, ...
FROM 테이블명1 JOIN 테이블명2 USING(속성명);
```



172 EQUI JOIN(동등 조인)



- JOIN에 참여하는 두 릴레이션의 속성 값을 비교하여 조건을 만족하는 튜플만 반환하는 조인을 THETA JOIN(세타 조인)이라고 하며, 조인에 사용되는 조건에는 =, ≠, <, ≤, >, ≥가 있다.
- EQUI JOIN(동등 조인)은 조인에 사용되는 조건 중 =(equal) 비교에 의해 같은 값을 가지는 행을 연결하여 결과를 생성하는 방법이다.
- EQUI JOIN의 결과 표시되는 속성(차수)은 두 릴레이션의 속성을 더한 것과 같다.
- EQUI JOIN에서 JOIN 조건이 '='일 때 동일한 속성이 두 번 나타나게 되는데, 이 중 중복된 속성을 제거하여 같은 속성을 한 번만 표기하는 방법을 NATURAL JOIN(자연 조인)이라고 한다.
- EQUI JOIN에서 연결 고리가 되는 공통 속성을 JOIN 속성이라고 한다.

- LEFT OUTER JOIN : INNER JOIN의 결과를 구한 후, 우측 향 릴레이션의 어떤 튜플과도 맞지 않는 좌측 향의 릴레이션에 있는 튜플들에 NULL 값을 붙여서 INNER JOIN의 결과에 추가함

- 표기 형식

```
SELECT [테이블명1.]속성명, [테이블명2.]속성명, ...
FROM 테이블명1 LEFT OUTER JOIN 테이블명2
ON 테이블명1.속성명 = 테이블명2.속성명;
```

- RIGHT OUTER JOIN : INNER JOIN의 결과를 구한 후, 좌측 향 릴레이션의 어떤 튜플과도 맞지 않는 우측 향의 릴레이션에 있는 튜플들에 NULL 값을 붙여서 INNER JOIN의 결과에 추가함

- 표기 형식

```
SELECT [테이블명1.]속성명, [테이블명2.]속성명, ...
FROM 테이블명1 RIGHT OUTER JOIN 테이블명2
ON 테이블명1.속성명 = 테이블명2.속성명;
```



173 OUTER JOIN



예제 <학생> 테이블과 <학과> 테이블에서 '학과코드' 값이 같은 튜플을 JOIN하여 '학번', '이름', '학과코드', '학과명'을 출력하는 SQL문을 작성하시오. 이때, '학과코드'가 입력되지 않은 학생도 출력하시오.

```
SELECT 학번, 이름, 학생.학과코드, 학과명
FROM 학생 LEFT OUTER JOIN 학과
ON 학생.학과코드 = 학과.학과코드;
```

해설 INNER JOIN을 하면 '학과코드'가 입력되지 않은 "박치민"은 출력되지 않는다. 그러므로 JOIN 구문을 기준으로 왼쪽 테이블, 즉 <학생>의 자료는 모두 출력되는 LEFT JOIN을 사용한 것이다. 다음과 같이 JOIN 구문을 기준으로 테이블의 위치를 교환하여 RIGHT JOIN을 사용해도 결과는 같다.

```
SELECT 학번, 이름, 학생.학과코드, 학과명
FROM 학과 RIGHT OUTER JOIN 학생
ON 학과.학과코드 = 학생.학과코드;
```

<결과>

학번	이름	학과코드	학과명
15	고길동	com	컴퓨터
16	이순신	han	국어
17	김선달	com	컴퓨터
19	아무개	han	국어
37	박치민		



- 트리거(Trigger)는 데이터베이스 시스템에서 데이터의 삽입(Insert), 갱신(Update), 삭제(Delete) 등의 이벤트(Event)가 발생할 때 관련 작업이 자동으로 수행되게 하는 절차형 SQL이다.
- 트리거는 데이터베이스에 저장되며, 데이터 변경 및 무결성 유지, 로그 메시지 출력 등의 목적으로 사용된다.

9장

소프트웨어 개발 보안 구축



440256

20.11, 필기 24.7, 23.5, 23.2, 22.7, 22.4, 21.3, 20.8, 20.6



175

소프트웨어 개발 보안 요소

A

소프트웨어 개발 보안 요소는 소프트웨어 개발에 있어 충족시켜야 할 요소 및 요건을 의미하며, 기밀성, 무결성, 가용성을 보안의 3대 요소라 한다.

필기 23.5, 22.7, 22.4, 21.3, 20.8

기밀성(Confidentiality)

- 시스템 내의 정보와 자원은 인가된 사용자에게만 접근이 허용된다.
- 정보가 전송 중에 노출되더라도 데이터를 읽을 수 없다.

필기 24.7, 23.5, 23.2, 22.7, 22.4, 21.3, 20.8, 20.6

무결성(Integrity)

시스템 내의 정보는 오직 인가된 사용자만 수정할 수 있다.

20.11, 필기 23.5, 22.7, 22.4, 21.3, 20.8

가용성(Availability)

인가받은 사용자는 시스템 내의 정보와 자원을 언제든지 사용할 수 있다.

인증(Authentication)

- 시스템 내의 정보와 자원을 사용하려는 사용자가 합법적인 사용자인지를 확인하는 모든 행위이다.
- 대표적 방법 : 패스워드, 인증용 카드, 지문 검사 등

부인 방지(NonRepudiation)

데이터를 송·수신한 자가 송·수신 사실을 부인할 수 없도록 송·수신 증거를 제공한다.

340329

20.7, 필기 24.2, 23.7, 21.8



176

SQL 삽입

A

- SQL 삽입(Injection)은 웹 응용 프로그램에 SQL을 삽입하여 내부 데이터베이스(DB) 서버의 데이터를 유출 및 변조하고, 관리자 인증을 우회하는 보안 약점이다.
- 동적 쿼리에 사용되는 입력 데이터에 예약어 및 특수문자가 입력되지 않게 필터링 되도록 설정하여 방지할 수 있다.

340336

필기 24.5, 21.5, 20.6



177

스택 가드

B

- 스택 가드(Stack Guard)는 널 포인터 역참조와 같이 주소가 저장되는 스택에서 발생하는 보안 약점을 막는 기술 중 하나이다.
- 메모리상에서 프로그램의 복귀 주소와 변수 사이에 특정 값을 저장한 후 그 값이 변경되었을 경우 오버플로우 상태로 판단하여 프로그램 실행을 중단함으로써 잘못된 복귀 주소의 호출을 막는다.

340337

필기 24.5, 24.2, 20.9, 20.6



178

접근 제어자

B

- 접근 제어자는 프로그래밍 언어에서 특정 개체를 선언할 때 외부로부터의 접근을 제한하기 위해 사용되는 예약어이다.
- 접근 제어자의 종류(접근 가능 : ○, 접근 불가능 : ×)

접근 제어자	클래스 내부	패키지 내부	하위 클래스	패키지 외부
필기 20.9, 20.6 Public	○	○	○	○
필기 20.6 Protected	○	○	○	×
필기 20.9 Default	○	○	×	×
필기 20.9, 20.6 Private	○	×	×	×

**179** 개인키 암호화 기법

- 개인키 암호화(Private Key Encryption) 기법은 동일한 키로 데이터를 암호화하고 복호화하는 암호화 기법이다.
- 대칭 암호 기법 또는 단일키 암호화 기법이라고도 한다.
- 암호화와 복호화 속도가 빠르다.
- 관리해야 할 키의 수가 많다.
- 개인키 암호화 기법의 종류에는 스트림 암호화 방식과 블록 암호화 방식이 있다.

스트림 암호화 방식	• 평문과 동일한 길이의 스트림을 생성하여 비트 단위로 암호화 하는 방식 • 종류 : LFSR, RC4, TKIP
블록 암호화 방식	• 한 번에 하나의 데이터 블록을 암호화 하는 방식 • 종류 : DES, SEED, AES, ARIA, IDEA, Skipjack

**180** IDEA

- IDEA(International Data Encryption Algorithm)는 스위스의 라이(Lai)와 메시(Messey)가 1990년에 개발한 PES를 개선한 알고리즘이다.
- 블록 크기는 64비트이고, 키 길이는 128비트이다.

**181** Skipjack

- Skipjack은 국가 안전 보장국(NSA)에서 개발한 암호화 알고리즘이다.
- 클리퍼 칩(Clipper Chip)이라는 IC 칩에 내장되어 있다.
- 블록 크기는 64비트이고, 키 길이는 80비트이다.
- 주로 음성 통신 장비에 삽입되어 음성 데이터를 암호화한다.

**182** DES

- DES(Data Encryption Standard)는 1975년 미국 NBS에서 발표한 개인키 암호화 알고리즘이다.
- 블록 크기는 64비트, 키 길이는 56비트이며 16회의 라운드를 수행한다.
- DES를 3번 적용하여 보안을 더욱 강화한 3DES(Triple DES)가 있다.

**183** AES

- AES(Advanced Encryption Standard)는 2001년 미국 표준 기술 연구소(NIST)에서 발표한 개인키 암호화 알고리즘이다.
- DES의 한계를 느낀 NIST에서 공모한 후 발표하였다.
- 블록 크기는 128비트이며, 키 길이에 따라 AES-128, AES-192, AES-256으로 분류된다.

**184** RSA

- RSA(Rivest Shamir Adleman)는 1978년 MIT의 라이베스트(Rivest), 샤미르(Shamir), 애들먼(Adelman)에 의해 제 안된 공개키 암호화 알고리즘이다.
- 큰 숫자를 소인수분해 하기 어렵다는 것에 기반하여 만들어졌다.

**185** TKIP

- TKIP(Temporal Key Integrity Protocol)는 기존의 무선 랜 보안 프로토콜인 WEP의 취약성을 보완한 데이터 보안 프로토콜이다.
- 암호 알고리즘의 입력 키 길이를 128비트로 늘리고 패킷당 키 할당, 키값 재설정 등의 키 관리 방식을 개선하였다.



186 해시

A

- 해시(Hash)는 임의의 길이의 입력 데이터나 메시지를 고정된 길이의 값이나 키로 변환하는 것을 의미한다.
- 해시 알고리즘을 해시 함수라고 부르며, 해시 함수로 변환된 값이나 키를 해시값 또는 해시키라고 부른다.
- 종류 : SHA 시리즈, HAVAL, MD4, MD5, N-NASH, SNEFRU 등



187 MD5

A

- MD5(Message Digest algorithm 5)는 1991년 R.Rivest가 MD4를 대체하기 위해 고안한 암호화 해시 함수이다.
- 블록 크기가 512비트이며, 키 길이는 128비트이다.



188 인증

B

- 인증(Authentication)은 다중 사용자 컴퓨터 시스템이나 네트워크 시스템에서 로그인을 요청한 사용자의 정보를 확인하고 접근 권한을 검증하는 보안 절차이다.
- 인증의 종류

종류	유형
지식 기반 인증 (Something You Know)	고정된 패스워드, 패스 프레이즈, 아이핀(i-Pin) 등
소유 기반 인증 (Something You Have)	신분증, 메모리 카드, 스마트 카드, OTP 등
생체 기반 인증 (Something You Are)	지문, 홍채/망막, 얼굴, 음성, 정맥 등
위치 기반 인증 (Somewhere You Are)	GPS, IP, 콜백 등
행위 기반 인증 (Something You Do)	서명, 동작 등



189 OAuth

A

- OAuth(Open Authorization)는 인터넷 애플리케이션에서 사용자 인증에 사용되는 표준 인증 방법으로, 공개 API(OpenAPI)로 구현되었다.
- 인터넷 사용자가 웹사이트나 애플리케이션에 비밀번호를 제공하지 않고 자신에게 접근 권한을 부여하여 사용할 수 있다.
- 2010년 IETF에서 1.0이 공식 표준안으로 발표되었다.



190 VPN

A

- VPN(Virtual Private Network, 가상 사설 통신망)은 인터넷 등 통신 사업자의 공중 네트워크와 암호화 기술을 이용하여 사용자가 마치 자신의 전용 회선을 사용하는 것처럼 해주는 보안 솔루션이다.
- SSL VPN : PC에 VPN Client 프로그램을 설치하여 VPN 서버에 접속하는 방식으로, 암호화를 위해 SSL 프로토콜을 사용함
- IPSec VPN : VPN 서버가 설치된 각각의 네트워크를 서로 연결하는 방식으로, 암호화를 위해 IPSec 프로토콜을 사용함



191 SIEM

A

- SIEM(Security Information and Event Management)은 다양한 장비에서 발생하는 로그 및 보안 이벤트를 통합하여 관리하는 빅 데이터 기반의 보안 솔루션이다.
- 방화벽, IDS, IPS, 웹 방화벽, VPN 등에서 발생한 로그 및 보안 이벤트를 통합하여 관리함으로써 비용 및 자원을 절약할 수 있다.
- 장기간의 로그 및 보안 이벤트를 수집 및 검색할 수 있는 빅데이터 기반의 통합 로그 수집 시스템이다.

440287



192 SSH

25.7, 23.4



- SSH(Secure SHell, 시큐어 셸)는 다른 컴퓨터에 원격으로 접속하여 작업을 수행할 수 있도록 다양한 기능을 지원하는 프로토콜 또는 이를 이용한 응용 프로그램이다.
- 데이터 암호화와 강력한 인증 방법으로 보안성이 낮은 네트워크에서도 안전하게 통신할 수 있다.
- 키(key)를 통한 인증 방법을 사용하려면 사전에 클라이언트의 공개키를 서버에 등록해야 한다.
- 기본적으로는 22번 포트를 사용한다.

340362



193 AAA(=3A)

21.10



- AAA는 다음 3가지 기능을 기반으로 사용자의 컴퓨터 자원 접근에 대한 처리와 서비스를 제공하는 기반 구조(Infrastructure) 또는 규격을 의미한다.
- 인증(Authentication) : 접근하는 사용자의 신원을 검증하는 기능
- 인가(Authorization) : 신원이 검증된 사용자에게 특정된 권한과 서비스를 허용하는 기능
- 과금(Accounting) : 사용자가 어떤 종류의 서비스를 이용했고, 얼마만큼의 자원을 사용했는지 기록 및 보관하는 기능

340363



194 정보보호 관리 체계

22.5



- 정보보호 관리 체계(ISMS; Information Security Management System)는 정보 자산을 안전하게 보호하기 위한 보호 절차와 대책을 의미한다.
- 조직에 맞는 정보보호 정책을 수립하고, 위협에 상시 대응하는 여러 보안 대책을 통합 관리·운용한다.
- 우리나라에서는 정보보호 관리 체계를 평가하고 인증하는 사업을 한국인터넷진흥원(KISA)에서 운영하고 있다.

340552



195 SYN Flooding

25.7



- TCP(Transmission Control Protocol)는 신뢰성 있는 전송을 위해 3-wayhandshake를 거친 후에 데이터를 전송하게 되는데, SYN Flooding은 공격자가 가상의 클라이언트로 위장하여 3-way-handshake 과정을 의도적으로 중단시킴으로써 공격 대상인 서버가 대기 상태에 놓여 정상적인 서비스를 수행하지 못하게 하는 공격 방법이다.
- 3-way-handshake
 - 신뢰성 있는 연결을 위해 송신지와 수신지 간의 통신에 앞서 3단계에 걸친 확인 작업을 수행한 후 통신을 수행한다.
 - 1단계 : 송신지에서 수신지로 'SYN' 패킷을 전송
 - 2단계 : 수신지에서 송신지로 'SYN + ACK' 패킷을 전송
 - 3단계 : 송신지에서 수신지로 'ACK' 패킷을 전송

340366



196 SMURFING

24.10, 필기 24.2, 23.5, 20.6



- SMURFING(스머핑)은 IP나 ICMP의 특성을 악용하여 엄청난 양의 데이터를 한 사이트에 집중적으로 보냄으로써 네트워크를 불능 상태로 만드는 공격 방법이다.
- SMURFING 공격을 무력화하는 방법 중 하나는 각 네트워크 라우터에서 브로드캐스트 주소를 사용할 수 없게 미리 설정해 놓는 것이다.

340369



197 LAND Attack

20.5, 필기 24.2, 23.2



LAND Attack(Local Area Network Denial Attack)은 패킷을 전송할 때 송신 IP 주소와 수신 IP 주소를 모두 공격 대상의 IP 주소로 하여 공격 대상에게 전송하는 것으로, 이 패킷을 받은 공격 대상은 송신 IP 주소가 자신이므로 자신에게 응답을 수행하게 되는데, 이러한 패킷이 계속해서 전송될 경우 자신에 대해 무한히 응답하게 하는 공격 방법이다.

340513

25.4



198 스케어웨어



스케어웨어(Scareware)는 사용자에게 컴퓨터가 바이러스에 감염되었다고 속이거나 보안 위험이 있다고 거짓 경고 메시지를 보내어 불안감을 조성한 후 문제 해결을 명목으로 불필요한 소프트웨어 구매를 유도하는 악성 소프트웨어이다.

340372

25.4, 21.4, 필기 24.7, 22.7, 21.3



199 세션 하이재킹



- 세션 하이재킹(Session Hijacking)은 상호 인증 과정을 거친 후 접속해 있는 서버와 서로 접속되어 클라이언트 사이의 세션 정보를 가로채는 공격 기법으로, 접속을 위한 인증 정보 없이도 가로챈 세션을 이용해 공격자가 원래의 클라이언트인 것처럼 위장하여 서버의 자원이나 데이터를 무단으로 사용한다.
- TCP 3-Way-Handshake 과정에 끼어들어서 클라이언트와 서버 간의 동기화된 시퀀스 번호를 가로채 서버에 무단으로 접근하는 TCP 세션 하이재킹이 대표적인 예이다.

340373

21.10



200 ARP 스푸핑



ARP 스푸핑(ARP Spoofing)은 ARP의 취약점을 이용한 공격 기법으로, 자신의 물리적 주소(MAC)를 공격 대상의 것으로 변조하여 공격 대상에게 도달해야 하는 데이터 패킷을 가로채거나 방해하는 기법이다.

※ ARP(Address Resolution Protocol) : 호스트의 IP 주소를 호스트와 연결된 네트워크 접속 장치의 물리적 주소(MAC Address)로 변환해주는 프로토콜

440300

22.10



201 사회 공학



사회 공학(Social Engineering)은 컴퓨터 보안에 있어서, 인간 상호 작용의 깊은 신뢰를 바탕으로 사람들을 속여 정상 보안 절차를 깨트리기 위한 비기술적 시스템 침입 수단이다.

440301

22.10



202 다크 데이터



다크 데이터(Dark Data)는 특정 목적을 가지고 데이터를 수집하였으나, 이후 활용되지 않고 저장만 되어 있는 대량의 데이터를 의미한다.

440302

22.10



203 타이포스쿼팅



- 타이포스쿼팅(Typosquatting)은 네티즌들이 사이트에 접속할 때 주소를 잘못 입력하거나 철자를 빠뜨리는 실수를 이용하기 위해 이와 유사한 유명 도메인을 미리 등록하는 것이다.
- URL 하이재킹(Hijacking)이라고도 한다.

340374

20.11, 필기 23.2



204 스니핑



스니핑(Sniffing)은 네트워크의 중간에서 남의 패킷 정보를 도청하는 해킹 유형의 하나로, 수동적 공격에 해당한다.



205 워터링 홀

(A)

- 워터링 홀(Watering Hole)은 목표 대상이 자주 방문하는 웹 사이트를 사전에 감염시켜 대상이 해당 사이트에 방문했을 때 악성 코드에 감염되게 하는 웹 기반 공격이다.
- 감염된 PC를 기반으로 대상이 속한 조직의 중요 시스템에 접근하거나 불능으로 만드는 등의 영향력을 행사할 수 있다.



206 키로거 공격

(C)

키로거 공격(Key Logger Attack)은 컴퓨터 사용자의 키보드 움직임을 탐지해 ID, 패스워드, 계좌번호, 카드번호 등과 같은 개인의 중요한 정보를 몰래 빼가는 해킹 공격이다.



207 랜섬웨어

(B)

랜섬웨어(Ransomware)는 인터넷 사용자의 컴퓨터에 잠입해 내부 문서나 파일 등을 암호화해 사용자가 열지 못하게 하는 프로그램으로, 암호 해독용 프로그램의 전달을 조건으로 사용자에게 돈을 요구하기도 한다.



208 백도어

(C)

- 백도어(Back Door, Trap Door)는 시스템 설계자가 서비스 기술자나 유지 보수 프로그램 작성자의 액세스 편의를 위해 시스템 보안을 제거하여 만들어놓은 비밀 통로로, 컴퓨터 범죄에 악용되기도 한다.
- 백도어 탐지 방법 : 무결성 검사, 열린 포트 확인, 로그 분석, SetUID 파일 검사 등



209 기타 정보 보안 관련 용어 1

(A)

24.4

Rootkit

- 시스템에 침입한 후 침입 사실을 숨긴 채 백도어, 트로이목마를 설치하고, 원격 접근, 내부 사용 흔적 삭제, 관리자 권한 획득 등 주로 불법적인 해킹에 사용되는 기능들을 제공하는 프로그램들의 모음이다.
- 컴퓨터의 운영체제에서 실행 파일과 실행 중인 프로세스를 숨김으로써 운영체제 검사 및 백신 프로그램의 탐지를 피할 수 있다.

24.4

APT(Advanced Persistent Threats, 지능형 지속 위협)

다양한 IT 기술과 방식들을 이용해 조직적으로 특정 기업이나 조직 네트워크에 침투해 활동 거점을 마련한 뒤 때를 기다리면서 보안을 무력화시키고 정보를 수집한 다음 외부로 빼돌리는 형태의 공격이다.

Ping of Death(죽음의 핑)

Ping 명령을 전송할 때 패킷의 크기를 인터넷 프로토콜 허용 범위 이상으로 전송하여 공격 대상의 네트워크를 마비시키는 서비스 거부 공격 방법이다.

큐싱(Qshing)

QR코드(Quick Response Code)를 통해 악성 앱의 다운로드를 유도하거나 악성 프로그램을 설치하도록 하는 금융사기 기법의 하나로, QR코드와 개인정보 및 금융정보를 낚는다(Fishing)는 의미의 합성 신조어이다.

스미싱(Smishing)

문자 메시지(SMS)를 이용해 사용자의 개인 신용 정보를 빼내는 수법이다.

스피어 피싱(Spear Phishing)

사회 공학의 한 기법으로, 특정 대상을 선정한 후 그 대상에게 일반적인 이메일로 위장한 메일을 지속적으로 발송하여, 발송 메일의 본문 링크나 첨부된 파일을 클릭하도록 유도해 사용자의 개인 정보를 탈취하는 공격이다.



210 기타 정보 보안 관련 용어 2 A

좀비(Zombie) PC

악성코드에 감염되어 다른 프로그램이나 컴퓨터를 조종하도록 만들어진 컴퓨터로, C&C(Command & Control) 서버의 제어를 받아 주로 DDoS 공격 등에 이용된다.

C&C 서버

해커가 원격지에서 감염된 좀비 PC에 명령을 내리고 악성코드를 제어하기 위한 용도로 사용하는 서버를 말한다.

봇넷(Botnet)

악성 프로그램에 감염되어 악의적인 의도로 사용될 수 있는 다수의 컴퓨터들이 네트워크로 연결된 형태를 말한다.

제로 데이 공격(Zero Day Attack)

보안 취약점이 발견되었을 때 발견된 취약점의 존재 자체가 널리 공표되기도 전에 해당 취약점을 통하여 이루어지는 보안 공격으로, 공격의 신속성을 의미한다.

23.4

트로이 목마(Trojan Horse)

정상적인 기능을 하는 프로그램으로 위장하여 프로그램 내에 숨어 있다가 해당 프로그램이 동작할 때 활성화되어 부작용을 일으키지만 자기 복제 능력은 없다.

CC(Common Criteria) 인증

국가마다 서로 다른 정보보호시스템 평가기준을 연동하고 평가결과를 상호 인증하기 위해 제정된 정보보안 평가기준으로, ISO/IEC 15408에 등록된 국제 표준이다.

멀버타이징(Malvertising)

악성 소프트웨어를 뜻하는 멀웨어(Malware)와 광고(Advertising)의 합성어로, 온라인 광고를 통해 악성코드를 유포시키는 행위를 말한다.

정보공유분석센터(ISAC; Information Sharing & Analysis Center)

취약점 및 침해요인과 그 대응방안에 관한 정보를 제공하며, 침해사고가 발생하는 경우 실시간 경보·분석체계를 운영하고, 금융·통신 등 분야별 정보통신기반시설을 보호하기 위한 업무를 수행하는 곳이다.

침입 탐지 시스템(IDS; Intrusion Detection System)

컴퓨터 시스템의 비정상적인 사용, 오용, 남용 등을 실시간으로 탐지하는 시스템이다.

데이터 디들링(Data Diddling)

처리할 자료를 다른 자료와 바꿔서 처리하는 것으로, 입력값이나 출력값을 부정한 의도로 수정하여 잘못된 결과가 나오도록 유도하는 방식이다.

비트로커(BitLocker)

Windows 7부터 지원되기 시작한 Windows 전용의 볼륨 암호화 기능으로, TPM(Trusted Platform Module)과 AES-128 알고리즘을 사용한다.

공급망 공격(Supply Chain Attack)

소프트웨어 공급망에 침투하여 악성코드를 배포하는 공격으로, SW 빌드 및 배포 과정에 악성코드를 삽입하여 이용자들을 공격한다.

23.4

바이러스(Virus)

컴퓨터의 정상적인 작동을 방해하기 위해 운영체제나 저장된 데이터에 손상을 입히는 프로그램으로, 자신을 복제할 수 있고, 다른 프로그램을 감염시킬 수 있다.

23.4, 필기 24.5, 22.7, 22.4

웜(Worm)

네트워크를 통해 연속적으로 자신을 복제하여 시스템의 부하를 높임으로써 결국 시스템을 다운시키는 바이러스의 일종으로, 분산 서비스 거부 공격, 버퍼 오버플로 공격, 슬래머 등이 웜 공격의 한 형태이다.